INTERVIEW PREPARATION GUIDE

100 React Native Interview Q&A

Theory · Strong Answers · Example Code · Architecture Diagrams

100	10	85+	6
Questions	Sections	Diagrams	Languages

Prepared for Mohammed Rinshad M I
React Native Engineer · iOS & Android · AI-Powered Mobile Apps · 2026

How to Use This Guide

This guide turns your resume into **100 realistic, interview-ready questions** across the full stack you actually work in — React Native, TypeScript, native modules, real-time, AI/LLM features, backend, payments and DevOps. Every question is answered in four layers so you understand it, not just memorise it:

THEORY	The concept and the *why* — what the interviewer is really probing for.
ANSWER	A natural, confident spoken answer you can deliver out loud. **Bold** terms are keywords to land.
CODE	A concise, correct example — what good looks like in real code.
ARCHITECTURE	An ASCII diagram of the flow or system, so you can draw it on a whiteboard.

PRO TIP

Read the **answers out loud**. Interviews reward fluent delivery, not perfect recall. When you give a number from your resume (35% latency cut, 30% faster delivery), have a one-line 'how I measured it' ready for the follow-up.

Contents

1 React Native Core & Fundamentals

Components, JSX, hooks, lists, layout, platform code and re-renders.

Q1–Q10

Q1 REACT NATIVE CORE & FUNDAMENTALS

What is React Native and how does it actually render native UI from JavaScript?

THEORY

React Native lets you build mobile apps using React and JavaScript while rendering real native UI components (not a WebView). Your JS code runs on a separate JS thread, and it communicates with the native side (the UI/main thread) to mount and update actual platform widgets like UIView on iOS and android.view.View on Android. Historically this happened over an asynchronous serialized bridge, while the newer architecture (Fabric, TurboModules, JSI) replaces it with a synchronous JSI layer for direct calls.

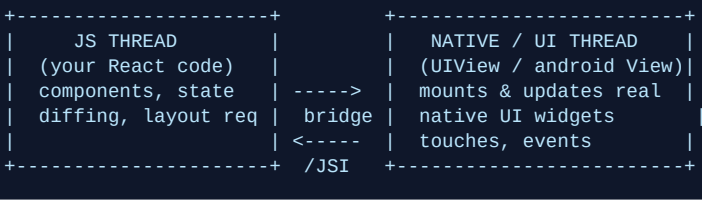
STRONG ANSWER

React Native lets us build cross-platform mobile apps using JavaScript. My JavaScript code runs on a separate JavaScript thread, while the actual UI is rendered using native components like UIView on iOS and android.view.View on Android. JavaScript doesn't draw the UI itself—it tells React Native what the UI should look like, and React Native creates or updates the corresponding native views. In the old architecture, communication happened through an asynchronous Bridge. In the new architecture, JSI, Fabric, and TurboModules remove that Bridge overhead, making communication faster and improving performance.

CODE · TypeScript / TSX

```tsx
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
// These JSX elements are NOT DOM nodes — they map to real native views.
// <View> -> UIView (iOS) / android.view.View (Android)
// <Text> -> UILabel (iOS) / TextView (Android)
export default function App(): JSX.Element {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello, native world</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  title: { fontSize: 20, fontWeight: '600' },
});
```

ARCHITECTURE / FLOW



Q2 REACT NATIVE CORE & FUNDAMENTALS

What are functional components and how does JSX work in React Native?

THEORY

A functional component is a plain JavaScript/TypeScript function that takes props and returns UI described in JSX. JSX is syntactic sugar: each tag compiles down to a `React.createElement` call that produces a lightweight element object describing what to render. Functional components are the modern default and, combined with Hooks, can hold state and side effects without needing classes.

STRONG ANSWER

A functional component is simply a JavaScript function that accepts props and returns JSX to describe the UI. JSX looks like HTML, but it's actually JavaScript syntax that React converts into React elements. In React Native, JSX uses native components like View, Text, and Image instead of HTML elements like div or span. I prefer functional components because they're simpler, easier to read, and with Hooks, they can manage state and lifecycle logic without needing class components. One important thing in React Native is that any text must be wrapped inside a Text component; otherwise, it will throw an error.

CODE · TypeScript / TSX

```tsx
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
type Props = { name: string; online?: boolean };
// A functional component: function in, JSX out.
export function UserBadge({ name, online = false }: Props) {
  return (
    <View style={styles.row}>
      {/* In RN, every piece of text must live inside <Text> */}
      <Text style={styles.name}>{name}</Text>
      <Text>{online ? 'I online' : 'I offline'}</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  name: { fontWeight: '600' },
});
```

Q3 REACT NATIVE CORE & FUNDAMENTALS

What is the difference between props and state in React Native?

THEORY

Props are read-only inputs passed from a parent to a child component; they flow downward and the child must not mutate them. State is data owned and managed *inside* a component that can change over time, and updating it (via a setter) triggers a re-render. The rule of thumb: props configure a component from the outside, state tracks things that change from within.

STRONG ANSWER

Props and state are both used to manage data in React Native, but they serve different purposes. Props are data passed from a parent component to a child component. They're read-only, so the child shouldn't modify them. State, on the other hand, is data that belongs to the component itself. When the state changes using the useState setter, React automatically re-renders the component to update the UI. In short, props are passed in from outside, while state is managed inside the component.

CODE · TypeScript / TSX

```tsx
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';
// Child receives data + callback as PROPS (read-only here)
function Counter({ count, onAdd }: { count: number; onAdd: () => void }) {
  return (
    <View>
      <Text>Count: {count}</Text>
      <Button title="Add" onPress={onAdd} />
    </View>
  );
}
// Parent OWNS the state and passes it down
export function Screen() {
  const [count, setCount] = useState(0); // state lives here
  return <Counter count={count} onAdd={() => setCount((c) => c + 1)} />;
}
```

ARCHITECTURE / FLOW

```
PARENT (owns state)
        +-------------------+
        |  count = 3        |
        |  setCount(...)    |
        +---------+---------+
          props |  (read-only, flows DOWN)
                v
        +-------------------+
        |  CHILD            |
        |  uses count       |
        |  calls onAdd()  --+--> triggers parent setState
        +-------------------+
```

Q4 REACT NATIVE CORE & FUNDAMENTALS

How do useState and useEffect work, and when does useEffect run?

THEORY

useState declares a piece of reactive state and returns the current value plus a setter; calling the setter schedules a re-render. useEffect runs side effects (data fetching, subscriptions, timers) *after* render, and its dependency array controls when it re-runs: empty array means once on mount, listed deps mean run when any changes, and no array means after every render. Returning a function from the effect provides a cleanup that runs before the next effect and on unmount.

STRONG ANSWER

useState is used to store and update a component's state. It returns the current value and a setter function. Whenever I update the state using the setter, React automatically re-renders the component with the new value. useEffect is used for side effects, such as fetching data, calling APIs, setting up timers, or subscribing to events. By default, it runs after the component renders. If I pass an empty dependency array ([]), it runs only once when the component mounts. If I pass dependencies, it runs whenever those values change. I also return a cleanup function when using timers or subscriptions to prevent memory leaks.

CODE · TypeScript / TSX

```tsx
import React, { useState, useEffect } from 'react';
import { Text } from 'react-native';
export function Clock() {
  const [now, setNow] = useState(() => new Date());
  useEffect(() => {
    // runs after mount because deps array is empty
```

CODE · TypeScript / TSX (continued)

```
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id); // cleanup on unmount
  }, []);
  return <Text>{now.toLocaleTimeString()}</Text>;
}
```

ARCHITECTURE / FLOW

```
render() ---> commit to screen ---> useEffect runs
   ^                                      |
   |                                      | (state/prop dep changes)
   |                                      v
   |          cleanup() <--- before next effect / on unmount
   +----------------------------------------+
deps []      -> run once on mount
deps [a, b]  -> run when a or b changes
deps omitted -> run after EVERY render
```

Q5 REACT NATIVE CORE & FUNDAMENTALS

What are useRef, useMemo, and useCallback, and when would you use each?

THEORY

useRef holds a mutable value that persists across renders without triggering re-renders, commonly used to reference native components (to call methods like focus or scrollTo) or to store mutable instance-like data. useMemo caches the result of an expensive computation and recomputes it only when its dependencies change. useCallback memoizes a function identity so it stays stable between renders, which matters when passing callbacks to memoized children or to dependency arrays.

STRONG ANSWER

useRef stores a value that persists across re-renders without causing a re-render. I commonly use it to access native components like a TextInput to call focus(), or to store values like timers or previous state. useMemo memoizes a computed value, so React only recalculates it when its dependencies change. I use it for expensive calculations or derived data. useCallback memoizes a function, so its reference stays the same between renders. I use it when passing callbacks to child components, especially those wrapped with React.memo, to avoid unnecessary re-renders.

CODE · TypeScript / TSX

```
import React, { useRef, useMemo, useCallback } from 'react';
import { TextInput, Button, View } from 'react-native';
export function SearchBox({ items }: { items: string[] }) {
  const inputRef = useRef<TextInput>(null); // ref to native node
  // expensive work memoized — recomputes only when items change
  const sorted = useMemo(() => [...items].sort(), [items]);
  // stable function identity across renders
  const focusInput = useCallback(() => inputRef.current?.focus(), []);
  return (
    <View>
      <TextInput ref={inputRef} placeholder={`${sorted.length} items`} />
      <Button title="Focus" onPress={focusInput} />
    </View>
  );
}
```

ARCHITECTURE / FLOW

```
useRef    -> persists value, NO re-render   (e.g. .current = node)
useMemo   -> caches a VALUE,  recompute on dep change
useCallback -> caches a FUNCTION identity,  stable across renders
         deps change?
   no  ->  return cached result
   yes ->  recompute & cache new result
```

Q6 REACT NATIVE CORE & FUNDAMENTALS

What is the difference between FlatList and ScrollView, and what is list virtualization?

THEORY

ScrollView renders *all* of its children at once, so it's fine for small, finite content but expensive and memory-heavy for long lists. FlatList is built for large datasets because it virtualizes: it only renders the items currently visible (plus a small buffer) and recycles them as you scroll, keeping memory and render cost low. FlatList also provides built-in features like pull-to-refresh, pagination via onEndReached, and item separators.

STRONG ANSWER

ScrollView renders all its child components at once, so it's best for small lists or screens with only a few items. FlatList is designed for large or dynamic lists. It uses list virtualization, which means it only renders the items visible on the screen and a few nearby items, improving performance and reducing memory usage. FlashList is an optimized alternative to FlatList from Shopify. It also uses virtualization but is faster and more memory-efficient, especially for very large lists. In production apps, I prefer FlashList when performance is critical, while FlatList is still a great built-in choice for most use cases. I also make sure to use a stable keyExtractor, and if items have a fixed height, I provide getItemLayout (for FlatList) to improve scrolling performance.

CODE · TypeScript / TSX

```tsx
import React from 'react';
import { FlatList, Text, View } from 'react-native';
type Item = { id: string; title: string };
export function MessageList({ data }: { data: Item[] }) {
  return (
    <FlatList
      data={data}
      keyExtractor={(item) => item.id}          // stable keys
      renderItem={({ item }) => (
        <View style={{ padding: 12 }}>
          <Text>{item.title}</Text>
        </View>
      )}
      initialNumToRender={10}
      onEndReachedThreshold={0.5}
      onEndReached={() => {/* load more */}}
    />
  );
}
```

ARCHITECTURE / FLOW

```
ScrollView: renders EVERYTHING (heavy for long lists)
[1][2][3][4][5][6][7][8][9][10]...[1000]  all in memory
FlatList (virtualized): only viewport + buffer rendered
        +-------- viewport --------+
  ...   | [4] [5] [6] [7] |   ...    off-screen = recycled
        +-------------------------+
   ^ recycled                recycled ^
```

Q7 REACT NATIVE CORE & FUNDAMENTALS

How does styling work with StyleSheet, and how do you build layouts with Flexbox in React Native?

THEORY

React Native styles are written as JavaScript objects, typically created with StyleSheet.create, using camelCased properties and unitless numbers that represent density-independent pixels. Layout uses Flexbox, but with key differences from the web: the default flexDirection is column (not row), and every View is a flex container. You position children with justifyContent (main axis), alignItems (cross axis), and size them with flex.

STRONG ANSWER

StyleSheet is used to organize and manage styles in React Native. It makes the code cleaner and can improve performance by creating optimized style objects. React Native styles use camelCase, and numeric values are treated as density-independent pixels (dp), so we don't specify units like px. For layouts, React Native uses Flexbox. The main difference from the web is that the default flexDirection is column, so components are stacked vertically by default. I use justifyContent to align items along the main axis, alignItems along the cross axis, and flex to control how components share available space. I also keep my styles outside the component so they aren't recreated on every render.

CODE · TypeScript / TSX

```tsx
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
export function Card() {
  return (
    <View style={styles.card}>
      <Text style={styles.label}>Total</Text>
      <Text style={styles.value}>$42.00</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',          // default would be 'column'
    justifyContent: 'space-between', // main axis
    alignItems: 'center',          // cross axis
    padding: 16,
    borderRadius: 8,
    backgroundColor: '#f2f2f2',
  },
  label: { color: '#666' },
  value: { fontWeight: '700' },
});
```

ARCHITECTURE / FLOW

```
flexDirection: 'row'   (main axis ->  horizontal)
+-------------------------------------------+
| [Total]           <space-between>   [$42] |
+-------------------------------------------+
  justifyContent = along MAIN axis (-->)
  alignItems     = along CROSS axis (|)
(default flexDirection in RN = 'column', stacks vertically)
```

Q8 REACT NATIVE CORE & FUNDAMENTALS

How do you write platform-specific code in React Native?

THEORY

React Native exposes the Platform module so you can branch at runtime: Platform.OS tells you ios or android, and Platform.select returns a value based on the platform. For larger divergences you can split files using the .ios.js / .android.js (or .tsx) extensions, and the bundler automatically picks the right one for each build. This keeps a shared codebase while letting you honor platform conventions.

STRONG ANSWER

In React Native, I write platform-specific code only when iOS and Android need different behavior. For small differences, like styles or UI tweaks, I use Platform.OS or Platform.select(). For larger differences, I create separate files like Component.ios.tsx and Component.android.tsx, and React Native automatically loads the correct one. My goal is to share as much code as possible and only write platform-specific code when it's really needed.

CODE · TypeScript / TSX

```tsx
import { Platform, StyleSheet } from 'react-native';
const styles = StyleSheet.create({
  header: {
    // runtime branch on the platform
    paddingTop: Platform.OS === 'ios' ? 44 : 24,
    ...Platform.select({
      ios: { shadowOpacity: 0.2, shadowRadius: 4 },
      android: { elevation: 4 },
      default: {},
    }),
  },
});
// File-based split (Metro auto-resolves the right one):
//   Button.ios.tsx   -> used on iOS
//   Button.android.tsx -> used on Android
// import Button from './Button';  // no extension needed
export { styles };
```

ARCHITECTURE / FLOW

```
import Button from './Button'
          |
     Metro bundler resolves by build target
        /                       \
   iOS build                  Android build
   Button.ios.tsx             Button.android.tsx
Runtime branch:  Platform.OS === 'ios' ? A : B
                 Platform.select({ ios, android, default })
```

Q9 REACT NATIVE CORE & FUNDAMENTALS

How do you handle touch interactions with Pressable, and what are the basics of navigation?

THEORY

For touch, modern React Native favors Pressable, a flexible primitive that exposes press states (pressed, hovered) and supports onPress, onLongPress, and styling based on interaction, superseding older components like TouchableOpacity. For moving between screens, the de-facto library is React Navigation, where you wrap the app in a NavigationContainer, define a navigator (such as a stack), and move around using the navigation prop's methods like navigate, push, and goBack.

STRONG ANSWER

Pressable is my preferred component for handling touch interactions because it's flexible and supports events like onPress, onLongPress, and a pressed state for visual feedback. For navigation, I use React Navigation. I wrap the app with NavigationContainer, define my navigators, and use the navigation object to move between screens with methods like navigate() and goBack(). If I need to pass data, I send it as route parameters and access it on the destination screen. In TypeScript projects, I also define navigation types to get better type safety and autocomplete.

CODE · TypeScript / TSX

```tsx
import React from 'react';
import { Pressable, Text } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
type Stack = { Home: undefined; Details: { id: string } };
const Nav = createNativeStackNavigator<Stack>();
function Home({ navigation }: any) {
  return (
    <Pressable
      onPress={() => navigation.navigate('Details', { id: '42' })}
      style={({ pressed }) => ({ opacity: pressed ? 0.6 : 1, padding: 16 })}
    >
      <Text>Go to details</Text>
    </Pressable>
  );
}
export function App() {
  return (
    <NavigationContainer>
      <Nav.Navigator>
        <Nav.Screen name="Home" component={Home} />
      </Nav.Navigator>
    </NavigationContainer>
  );
}
```

ARCHITECTURE / FLOW

```
NavigationContainer
    |
    +-- Stack.Navigator
          |
          +-- Screen 'Home'  --navigation.navigate('Details',{id})-->
          +-- Screen 'Details'  (reads route.params.id)
                          <-- navigation.goBack()
```

Q10 REACT NATIVE CORE & FUNDAMENTALS

Why are keys important in lists, what causes re-renders, and what is the component lifecycle in function components?

THEORY

Keys give React a stable identity for each item in a list so it can match elements between renders and update efficiently instead of re-creating them; using an array index as a key is risky when the list can reorder or change. A component re-renders when its state changes, its props change, or its parent re-renders, and React then reconciles the virtual tree to update only what changed. In function components the lifecycle (mount, update, unmount) is expressed through Hooks like useEffect rather than class methods such as componentDidMount.

STRONG ANSWER

Keys help React identify each item in a list, so it knows which items have changed, been added, or removed. That's why I always use a unique and stable ID instead of the array index whenever possible. A component re-renders when its state changes, when its props change, or when its parent re-renders. React then compares the new UI with the previous one and updates only what's necessary. In function components, the lifecycle is handled with Hooks. The component mounts, updates when its state or props change, and unmounts when it's removed. I use useEffect to handle side effects and return a cleanup function when needed, such as for timers or event listeners.

CODE · TypeScript / TSX

```tsx
import React, { useEffect } from 'react';
import { FlatList, Text } from 'react-native';
type Todo = { id: string; text: string };
function Row({ item }: { item: Todo }) {
  useEffect(() => {
    // 'mount' for this row
    return () => {/* 'unmount' cleanup */};
  }, []);
  return <Text>{item.text}</Text>;
}
export function TodoList({ todos }: { todos: Todo[] }) {
  return (
    <FlatList
      data={todos}
      // STABLE id, not the array index, so reorders patch correctly
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => <Row item={item} />}
    />
  );
}
```

ARCHITECTURE / FLOW

```
Re-render triggers:  state change | new props | parent re-render
        |
        v
   reconcile virtual tree --> patch only what differs
Function component lifecycle via Hooks:
  mount   -> useEffect(() => {...}, [])
  update  -> useEffect(() => {...}, [dep])
  unmount -> return () => {...}  (cleanup)
Keys: stable id  -> matched & reused   |  index -> bugs on reorder
2
TypeScript & JavaScript (ES6+)
Types vs interfaces, generics, utility types, async, and the JS to TS migration.
Q11–Q20
2
TypeScript & JavaScript (ES6+)
Types vs interfaces, generics, utility types, async, and the JS to TS migration.
Q11–Q20
```

ARCHITECTURE / FLOW (continued)

```
2
TypeScript & JavaScript (ES6+)
Types vs interfaces, generics, utility types, async, and the JS to TS migration.
Q11–Q20
```


```
2
TypeScript & JavaScript (ES6+)
Types vs interfaces, generics, utility types, async, and the JS to TS migration.
Q11–Q20
```

2 TypeScript & JavaScript (ES6+)

Types vs interfaces, generics, utility types, async, and the JS to TS migration.

Q11–Q20

Q11 TYPESCRIPT & JAVASCRIPT (ES6+)

Why do we use TypeScript in a React Native project and what problems does it solve over plain JavaScript?

THEORY

TypeScript is a typed superset of JavaScript that adds static types checked at compile time, then strips them out to produce plain JS. The core problem it solves is catching whole categories of bugs before runtime such as accessing undefined properties, passing wrong argument types, or misspelling fields. It also powers editor features like autocomplete, safe refactoring, and inline documentation.

STRONG ANSWER

I use TypeScript because it helps catch errors during development instead of at runtime. It provides static typing, so I can catch issues like passing the wrong props, accessing undefined values, or using the wrong data types before the app is deployed. It also improves code readability, makes refactoring safer, and provides better autocomplete and IntelliSense in the editor. Although it requires writing some type definitions, it saves time by reducing bugs and making large React Native projects easier to maintain.

CODE · TypeScript

```
// JavaScript: bug only shows up at runtime
function getName(user) {
  return user.profile.name; // crashes if profile is undefined
}
// TypeScript: error caught while typing
interface User {
  profile?: { name: string };
}
function getNameTs(user: User): string {
  // TS error: 'profile' is possibly 'undefined'
  // return user.profile.name;
  return user.profile?.name ?? "Guest"; // safe
}
const u: User = {};
console.log(getNameTs(u)); // "Guest", no crash
```

ARCHITECTURE / FLOW

```
JavaScript:  write -> run -> CRASH on device (runtime bug)
TypeScript:  write -> tsc type-check -> FAIL FAST in editor/CI
                        |
                        +--> emit plain JS -> run (fewer bugs)
```

Q12 TYPESCRIPT & JAVASCRIPT (ES6+)

What is the difference between type and interface in TypeScript, and when do you use each?

THEORY

Both type aliases and interface can describe the shape of an object, and for most object shapes they are interchangeable. The key differences: interface supports declaration merging (reopening to add fields) and is idiomatic for object and class contracts, while type can also alias unions, tuples, primitives, and mapped types. type uses intersection with the and operator to combine, whereas interface uses extends.

STRONG ANSWER

interface and type are both used to define types in TypeScript, but they have slightly different use cases. I usually use interface to define the shape of objects, such as component props or API responses, because it's easy to extend. I use type when I need more flexibility, like defining unions, tuples, function types, or primitive aliases. In most React Native projects, either works for component props, but I stay consistent with the project's coding standards.

CODE · TypeScript

```typescript
// interface: object shape, extendable
interface Button {
  label: string;
}
interface Button {
  onPress: () => void; // declaration merging
}
interface IconButton extends Button {
  icon: string;
}
// type: unions, tuples, primitives - interface can't do these
type Status = "idle" | "loading" | "error";
type Point = [number, number];
type ID = string | number;
// composing with type
type Card = Button & { elevated: boolean };
const b: IconButton = { label: "Go", onPress: () => {}, icon: "x" };
```

ARCHITECTURE / FLOW

```
interface              | type alias
-----------------------+----------------------------
object shapes      ok  | object shapes      ok
extends            ok  | & intersection     ok
declaration merge  ok  | declaration merge  NO
unions/tuples      NO  | unions/tuples      ok
primitive alias    NO  | primitive alias    ok
```

Q13 TYPESCRIPT & JAVASCRIPT (ES6+)

What are generics in TypeScript and why are they useful?

THEORY

Generics let you write reusable, type-safe code that works over many types by using a type parameter (commonly T) that the caller fills in. Instead of locking a function or component to one concrete type or falling back to any, the type flows through and is preserved on the output. This keeps full type checking and autocomplete while staying flexible.

STRONG ANSWER

Generics let me write reusable code while keeping full type safety. Instead of creating separate functions or components for different data types, I can write one generic version that works with any type. The caller specifies the type, and TypeScript ensures the correct type is used throughout. I also use constraints with extends when I need a generic type to have specific properties, such as an id field.

CODE · TypeScript

```
// generic function: T is inferred from the argument
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
const n = first([1, 2, 3]);      // n: number | undefined
const s = first(["a", "b"]);     // s: string | undefined
// generic with a constraint
interface HasId { id: string; }
function indexById<T extends HasId>(items: T[]): Record<string, T> {
  return items.reduce((acc, item) => {
    acc[item.id] = item;
    return acc;
  }, {} as Record<string, T>);
}
const map = indexById([{ id: "a", name: "Sam" }]);
// map["a"].name is fully typed
```

Q14 TYPESCRIPT & JAVASCRIPT (ES6+)

Which built-in utility types do you use most (Partial, Pick, Omit, Record, Readonly) and what do they do?

THEORY

Utility types transform existing types instead of redefining them, keeping a single source of truth. Partial of T makes all fields optional, Pick of T and keys selects a subset, Omit of T and keys removes a subset, Record of K and V builds an object map type, and Readonly of T makes fields immutable. Using them avoids duplicated, drifting type definitions.

STRONG ANSWER

I use utility types to make my code more reusable and avoid duplicating type definitions. Partial makes all properties optional, which is useful for update requests. Pick creates a type with only the properties I need. Omit creates a type by removing specific properties. Record creates an object with predefined keys and value types, like mapping a status to a color. Readonly makes properties immutable so they can't be changed accidentally. These utility types help keep the code clean, reduce duplication, and make it easier to maintain.

CODE · TypeScript

```
interface User {
  id: string;
  name: string;
  email: string;
  age: number;
}
// only some fields needed for an update
type UserUpdate = Partial<User>;
// derive a public profile from the model
type PublicUser = Pick<User, "id" | "name">;
// everything except the id
type NewUser = Omit<User, "id">;
// keyed map from a union to a value
```

CODE · TypeScript (continued)

```
type Status = "active" | "banned";
type StatusLabels = Record<Status, string>;
// immutable config
const config: Readonly<{ apiUrl: string }> = { apiUrl: "/v1" };
// config.apiUrl = "x"; // TS error: read-only
```

ARCHITECTURE / FLOW

```
User { id, name, email, age }
   |
   +-- Partial<User>  -> { id?, name?, email?, age? }
   +-- Pick<"id","name"> -> { id, name }
   +-- Omit<"id">       -> { name, email, age }
   +-- Readonly<User>   -> all fields read-only
Record<"active"|"banned", string> -> { active: string;
                                       banned: string }
```

Q15 TYPESCRIPT & JAVASCRIPT (ES6+)

What are union and discriminated union types, and how does TypeScript narrow them?

THEORY

A union type means a value can be one of several types, written with the pipe symbol. A discriminated (tagged) union adds a shared literal field, like a kind or status property, so TypeScript can tell the variants apart. Narrowing is when checks like typeof, in, or comparing the discriminant field let the compiler refine the type inside a branch, unlocking the right fields.

STRONG ANSWER

A union type allows a variable to have more than one type. For example, a value can be a string or a number. A discriminated union is a union where each type has a common property, such as status or type. TypeScript uses that property to automatically determine which type you're working with. This is called type narrowing. I commonly use discriminated unions to represent API or UI states like loading, success, and error. It makes the code safer because TypeScript knows which properties are available in each state and helps prevent runtime errors.

CODE · TypeScript

```
type RequestState =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: string[] }
  | { status: "error"; message: string };
function render(state: RequestState): string {
  switch (state.status) {
    case "idle":
      return "Tap to load";
    case "loading":
      return "Loading...";
    case "success":
      return state.data.join(", "); // data is known here
    case "error":
      return state.message;         // message is known here
  }
}
```

ARCHITECTURE / FLOW

```
RequestState (discriminant = status)
        |
 switch (state.status)
   |       |          |            |
 idle   loading   success       error
  ()      ()    {data:[]}   {message:string}
                  ^narrowed: only valid fields visible
```

Q16 TYPESCRIPT & JAVASCRIPT (ES6+)

What is the difference between Promises and async/await, and how do you handle errors with each?

THEORY

Promises and async/await are two syntaxes over the same underlying mechanism for asynchronous work. Promises use .then and .catch chaining, while async/await lets you write asynchronous code that reads top-to-bottom like synchronous code. With Promises you handle errors via .catch; with async/await you use try/catch, which also catches errors thrown from awaited calls.

STRONG ANSWER

Promises and async/await are both used to handle asynchronous operations like API calls. async/await is built on top of Promises, but it makes the code easier to read and write. That's why I usually prefer async/await. With Promises, I handle errors using .catch(). With async/await, I use a try...catch block. If I need to run multiple independent API calls at the same time, I use Promise.all() to improve performance instead of awaiting them one by one.

CODE · TypeScript

```typescript
// Promise style
function loadUser(id: string): Promise<User> {
  return fetch(`/users/${id}`)
    .then((res) => res.json())
    .catch((err) => {
      console.error(err);
      throw err;
    });
}
// async/await style with try/catch
async function loadUserAsync(id: string): Promise<User | null> {
  try {
    const res = await fetch(`/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return (await res.json()) as User;
  } catch (err) {
    console.error("Failed to load user", err);
    return null;
  }
}
// parallel, not sequential
async function loadTwo() {
  return Promise.all([loadUserAsync("1"), loadUserAsync("2")]);
}
interface User { id: string; name: string; }
```

Q17 TYPESCRIPT & JAVASCRIPT (ES6+)

Can you explain closures and how var, let, and const differ in scoping?

THEORY

A closure is a function that remembers the variables from the scope where it was created, even after that outer scope has returned. var is function-scoped and hoisted, while let and const are block-scoped and live in the temporal dead zone until declared. const prevents reassignment of the binding, though the referenced object can still be mutated.

STRONG ANSWER

A closure is when a function remembers and can access variables from its outer scope, even after the outer function has finished executing. In React Native, closures are common in event handlers, callbacks, and Hooks like useEffect. That's also why stale closure issues can happen if dependencies aren't managed correctly. For variable declarations, I use const by default because it can't be reassigned. I use let when I need to update a variable. I avoid var because it's function-scoped and hoisted, which can lead to unexpected behavior. let and const are block-scoped, making the code safer and easier to understand.

CODE · TypeScript

```
// closure: counter remembers `count`
function makeCounter() {
  let count = 0;
  return () => ++count; // closes over count
}
const next = makeCounter();
next(); next(); // 1, 2
// var vs let in a loop
const varFns: (() => number)[] = [];
for (var i = 0; i < 3; i++) varFns.push(() => i);
console.log(varFns.map((f) => f())); // [3, 3, 3]
const letFns: (() => number)[] = [];
for (let j = 0; j < 3; j++) letFns.push(() => j);
console.log(letFns.map((f) => f())); // [0, 1, 2]
const obj = { n: 1 };
obj.n = 2;       // ok, mutation allowed
// obj = {};     // error, reassignment blocked
```

ARCHITECTURE / FLOW

```
var  -> function-scoped, hoisted -> loop shares 1 binding
        [i][i][i] all point to final i (=3)
let  -> block-scoped per iteration
        [j=0][j=1][j=2] each callback keeps its own
```

Q18 TYPESCRIPT & JAVASCRIPT (ES6+)

How do you use destructuring, spread/rest, optional chaining, and nullish coalescing in everyday code?

THEORY

Destructuring pulls fields out of objects or arrays into variables, often with defaults. The spread operator copies or merges objects and arrays immutably, while rest collects remaining items into one variable. Optional chaining with question-mark-dot short-circuits to undefined instead of throwing on a missing path, and nullish coalescing with double question marks supplies a fallback only when the value is null or undefined.

STRONG ANSWER

I use these JavaScript features every day because they make the code cleaner and easier to maintain. Destructuring helps me extract values from objects or arrays, such as props or the return value from Hooks. Spread (...) lets me copy or update objects and arrays immutably, which is important for React state updates. Rest (...) collects the remaining properties or function arguments into a single object or array. Optional chaining (?.) lets me safely access nested properties without worrying about null or undefined. Nullish coalescing (??) provides a default value only when the value is null or undefined, unlike ||, which also treats 0, false, and an empty string as missing values.

CODE · TypeScript

```
interface Props {
  title: string;
  count?: number;
  user?: { address?: { city: string } };
}
function Banner({ title, count = 0, user }: Props) {
  // optional chaining: no crash if address is missing
  const city = user?.address?.city ?? "Unknown";
  // nullish vs OR: 0 stays 0 with ??, but OR would drop it
  const shown = count ?? 0;
  return `${title} (${shown}) - ${city}`;
}
// spread to update immutably, rest to split
const base = { theme: "dark", fontSize: 14 };
const updated = { ...base, fontSize: 16 };
const [head, ...tail] = [1, 2, 3, 4]; // head=1, tail=[2,3,4]
```

Q19 TYPESCRIPT & JAVASCRIPT (ES6+)

How do map, filter, and reduce work, and why do they matter for immutability in React?

THEORY

map transforms each element into a new array of the same length, filter keeps only elements passing a predicate, and reduce folds an array down to a single accumulated value. Crucially, all three return new arrays/values and never mutate the original. That immutability is what lets React detect changes by reference and re-render correctly.

STRONG ANSWER

map, filter, and reduce are array methods that I use frequently in React Native. map creates a new array by transforming each item. I often use it to render lists or update a specific item in state. filter creates a new array by removing items that don't match a condition. reduce processes all items and returns a single value, such as a total, a grouped object, or a lookup map. These methods are important because they don't mutate the original array. Instead, they return a new array, which helps React detect state changes and trigger re-renders correctly.

CODE · TypeScript

```
interface Todo { id: string; done: boolean; text: string; }
const todos: Todo[] = [
  { id: "1", done: false, text: "Buy milk" },
  { id: "2", done: true, text: "Walk dog" },
];
// map: immutable update of one item
const toggled = todos.map((t) =>
  t.id === "1" ? { ...t, done: true } : t
);
// filter: remove completed
const active = todos.filter((t) => !t.done);
// reduce: count remaining
const openCount = todos.reduce(
  (sum, t) => (t.done ? sum : sum + 1),
```

CODE · TypeScript (continued)

```
  0
);
console.log(active.length, openCount); // original todos untouched
```

ARCHITECTURE / FLOW

```
[a, b, c]
   |  map(fn)    -> [fn(a), fn(b), fn(c)]   (same length)
   |  filter(p)  -> [a, c]                   (subset)
   |  reduce(f,0)-> single value             (fold)
All return NEW data -> reference changes -> React re-renders
```

Q20 TYPESCRIPT & JAVASCRIPT (ES6+)

How would you migrate a live JavaScript React Native codebase to TypeScript without halting feature work?

THEORY

The realistic approach is incremental migration, not a big-bang rewrite. You enable TypeScript with allowJs so JS and TS coexist, rename files to .ts/.tsx file by file, and turn on strict mode gradually. Priority is given to shared, high-risk code first: API layers, models, and utilities, locking down type-safe API contracts so data shapes are trusted everywhere downstream.

STRONG ANSWER

I would migrate the project gradually instead of trying to convert everything at once. First, I'd configure TypeScript so JavaScript files continue to work, allowing the team to keep shipping features. Then I'd convert files one by one, starting with shared code like API services, utility functions, and common components because they provide the biggest benefit. At the beginning, I'd keep the TypeScript configuration less strict to make the migration smoother. As more of the project is converted, I'd gradually enable stricter type checking and replace temporary any types with proper types. This approach minimizes risk while improving code quality over time.

CODE · TypeScript

```
// tsconfig.json (phased strictness)
{
  "compilerOptions": {
    "allowJs": true,        // JS and TS coexist
    "checkJs": false,
    "strict": false,        // flip flags on gradually
    "strictNullChecks": true, // enable first - highest value
    "noImplicitAny": false,   // tighten later
    "jsx": "react-native"
  }
}
// type-safe API contract: define once, trust everywhere
export interface UserDTO {
  id: string;
  name: string;
  email: string | null;
}
export async function getUser(id: string): Promise<UserDTO> {
  const res = await fetch(`/users/${id}`);
  return (await res.json()) as UserDTO; // ideally validate with zod
}
```

ARCHITECTURE / FLOW

```
Big-bang rewrite  -> BLOCKS features, high risk  (avoid)
Incremental migration:
  1. tsconfig allowJs (JS + TS coexist)
  2. types/ + API contracts (models first)
  3. rename .js -> .ts, leaf modules outward
  4. enable strict flags one at a time
         strictNullChecks -> noImplicitAny -> strict
  5. zod validation at API boundary (runtime == types)
3
State Management & Data Fetching
Context, Redux Toolkit, thunks, selectors, React Query caching & mutations.
Q21–Q30
3
State Management & Data Fetching
Context, Redux Toolkit, thunks, selectors, React Query caching & mutations.
Q21–Q30
3
State Management & Data Fetching
Context, Redux Toolkit, thunks, selectors, React Query caching & mutations.
Q21–Q30

3
State Management & Data Fetching
Context, Redux Toolkit, thunks, selectors, React Query caching & mutations.
Q21–Q30
```

3 State Management & Data Fetching

Context, Redux Toolkit, thunks, selectors, React Query caching & mutations.

Q21–Q30

Q21 STATE MANAGEMENT & DATA FETCHING

What is the difference between local and global state, and when do you actually need a state management library?

THEORY

Local state lives inside a single component (via useState or useReducer) and is only needed by that component or its children. Global state is data many unrelated screens need, like the logged-in user, theme, or cart. You only reach for a library like Redux or Zustand when prop drilling becomes painful or when shared state changes frequently across the app.

STRONG ANSWER

Local state is data that's used only within a single component, like an input value, a loading indicator, or whether a modal is open. I usually manage it with useState. Global state is data that's shared across multiple screens or components, such as the logged-in user, app theme, or authentication status. I keep state local whenever possible because it's simpler. If the same data needs to be shared across many components or I'm passing props through multiple levels (prop drilling), then I use a state management solution like Context, Redux, or Zustand.

CODE · TypeScript / TSX

```tsx
// Local state - stays in the component
function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  // only this component cares about these values
  return <TextInput value={email} onChangeText={setEmail} />;
}
// Global state - many screens need the logged-in user
// Profile, Settings, Header all read auth.user
// -> a good candidate for Context, Redux or Zustand
```

ARCHITECTURE / FLOW

```
Local state:        Global state:
[Component]         [Store / Context]
  | useState          /     |     \
  v               Header  Cart  Profile
(self only)       (shared across screens)
```

Q22 STATE MANAGEMENT & DATA FETCHING

How does the Context API work and what are its main limitations for state management?

THEORY

The Context API lets you pass data through the tree without prop drilling using a Provider and useContext. Its biggest limitation is re-renders: when the context value changes, every component that consumes that context re-renders, even if it only uses a small part of the value. It also has no built-in tooling for async logic, devtools, or memoized selectors, so it works best for low-frequency data like theme or auth, not fast-changing state.

STRONG ANSWER

The Context API lets me share data across multiple components without passing props through every level. I create a Provider at the top of the component tree, and any child component can access the data using useContext(). It's great for things like themes, authentication, or language settings. One limitation is that when the context value changes, all components using that context may re-render, even if they only use part of the data. Because of that, I use Context for simple, infrequently changing global state. For more complex or frequently updated state, I prefer libraries like Redux Toolkit or Zustand.

CODE · TypeScript / TSX

```tsx
type AuthCtx = { user: User | null; login: (u: User) => void };
const AuthContext = createContext<AuthCtx | null>(null);
export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  // value changes -> ALL consumers re-render, even theme-only ones
  const value = useMemo(() => ({ user, login: setUser }), [user]);
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}
export const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be inside AuthProvider');
  return ctx;
};
```

ARCHITECTURE / FLOW

```
Provider value changes
        |
        v
  +-----------+-----------+
  v           v           v
ConsumerA  ConsumerB  ConsumerC
(all re-render, even if unused)
```

Q23 STATE MANAGEMENT & DATA FETCHING

Can you explain the core building blocks of Redux Toolkit: store, slice, reducers, and actions?

THEORY

Redux Toolkit (RTK) is the standard, modern way to write Redux with far less boilerplate. A slice bundles a piece of state with its reducers and auto-generates action creators. Reducers are pure functions that take state and an action and return the next state, and RTK lets you write 'mutating' logic safely using Immer. The store is the single source of truth that holds all slices combined.

STRONG ANSWER

Redux Toolkit is the recommended way to use Redux because it reduces boilerplate and makes state management simpler. The store is the central place where the application's global state is stored. A slice represents a feature, such as authentication or user data, and contains its state, reducers, and actions. Reducers define how the state should change, and actions are dispatched to trigger those reducers. In my components, I use useSelector to read data from the store and useDispatch to dispatch actions that update the state. Redux Toolkit uses Immer internally, so I can write reducers in a simple mutable style while keeping the state immutable.

CODE · TypeScript

```
import { createSlice, configureStore, PayloadAction } from '@reduxjs/toolkit';
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },          // Immer
    addBy: (state, action: PayloadAction<number>) => {
      state.value += action.payload;
    },
  },
});
export const { increment, addBy } = counterSlice.actions;
export const store = configureStore({
  reducer: { counter: counterSlice.reducer },
});
export type RootState = ReturnType<typeof store.getState>;
```

ARCHITECTURE / FLOW

```
Component               Store
   |  dispatch(action)     |
   +---------------------> reducer(state, action)
                             |
                           new state
                             |
   useSelector  <------------ +
   (re-render)
```

Q24 STATE MANAGEMENT & DATA FETCHING

How do you handle asynchronous logic like API calls in Redux Toolkit using createAsyncThunk?

THEORY

createAsyncThunk is RTK's helper for async flows like fetching data. It dispatches three lifecycle actions automatically: pending, fulfilled, and rejected. You handle these inside the slice's extraReducers to update loading, data, and error state. This keeps async logic out of components and gives you consistent loading and error handling.

STRONG ANSWER

createAsyncThunk is used in Redux Toolkit to handle asynchronous operations like API calls. It automatically creates three states: pending, fulfilled, and rejected. In extraReducers, I handle each state: pending → Set a loading state. fulfilled → Store the API response. rejected → Store the error message. In the component, I simply dispatch the thunk and use useSelector to read the loading state, data, and any errors from the Redux store. This keeps the component clean and makes API handling predictable.

CODE · TypeScript

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';
export const fetchUser = createAsyncThunk('user/fetch', async (id: string) => {
  const res = await fetch(`/api/users/${id}`);
  return (await res.json()) as User;
});
const userSlice = createSlice({
  name: 'user',
  initialState: { data: null as User | null, loading: false, error: '' },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUser.pending, (s) => { s.loading = true; s.error = ''; })
      .addCase(fetchUser.fulfilled, (s, a) => { s.loading = false; s.data = a.payload; })
```

CODE · TypeScript (continued)

```
      .addCase(fetchUser.rejected, (s, a) => {
        s.loading = false; s.error = a.error.message ?? 'Failed';
      });
  },
});
export default userSlice.reducer;
```

ARCHITECTURE / FLOW

```
dispatch(fetchUser(id))
        |
        v
  [pending]  -> loading = true
        |
   API call
     /     \
[fulfilled] [rejected]
 data=...    error=...
```

Q25 STATE MANAGEMENT & DATA FETCHING

What are selectors and why is memoization important when using useSelector?

THEORY

A selector is a function that reads a specific piece of state from the store, keeping that knowledge in one place. With useSelector, the component re-renders whenever the selected value changes by reference, so returning a new object or array each time causes needless re-renders. createSelector from Reselect memoizes derived data so it only recomputes when its inputs actually change. This keeps expensive calculations and re-renders under control.

STRONG ANSWER

A selector is a function that reads specific data from the Redux store. Instead of accessing the store directly in every component, I use selectors to keep the code clean and reusable. Memoization is important because useSelector re-renders a component when the selected value changes. If a selector creates a new object or array on every render, it can cause unnecessary re-renders. To avoid that, I use createSelector from Reselect, which memoizes the result and only recalculates it when the input data actually changes. This improves performance, especially for filtered or sorted data.

CODE · TypeScript / TSX

```
import { createSelector } from '@reduxjs/toolkit';
import { useSelector } from 'react-redux';
import type { RootState } from './store';
const selectTodos = (s: RootState) => s.todos.items;
const selectFilter = (s: RootState) => s.todos.filter;
// memoized: recomputes only when items or filter change
export const selectVisibleTodos = createSelector(
  [selectTodos, selectFilter],
  (items, filter) => items.filter((t) => t.status === filter)
);
function TodoList() {
  // stable reference -> no re-render unless inputs change
  const todos = useSelector(selectVisibleTodos);
  return <FlatList data={todos} renderItem={({ item }) => <Row t={item} />} />;
}
```

ARCHITECTURE / FLOW

```
inputs change?
   no -> return cached result (no re-render)
   yes -> recompute -> cache -> return
selectItems --\
              > createSelector -> visibleTodos
selectFilter -/
```

Q26 STATE MANAGEMENT & DATA FETCHING

What is React Query (TanStack Query) and how does server state differ from client state?

THEORY

React Query manages server state, which is data that lives on a backend and you only borrow, like a list of products fetched from an API. This differs from client state, which your app fully owns, like a toggle or form input. Server state is asynchronous, can become stale, and may be shared by many users, so it needs caching, background refetching, and synchronization that tools like Redux were never designed for. React Query handles fetching, caching, and updating that remote data for you.

STRONG ANSWER

React Query (TanStack Query) is a library for managing server state, which is data fetched from an API. It automatically handles data fetching, caching, loading states, error handling, and refetching, so I don't have to implement that logic myself. Server state is different from client state because it comes from the backend, can change at any time, and needs to stay in sync with the server. Client state is data that belongs to the app itself, such as theme, authentication status, or whether a modal is open. I use React Query for API data and Redux or Zustand for application state.

CODE · TypeScript / TSX

```tsx
import { useQuery } from '@tanstack/react-query';
async function getProducts(): Promise<Product[]> {
  const res = await fetch('/api/products');
  if (!res.ok) throw new Error('Network error');
  return res.json();
}
function ProductList() {
  const { data, isLoading, isError, refetch } = useQuery({
    queryKey: ['products'],
    queryFn: getProducts,
  });
  if (isLoading) return <ActivityIndicator />;
  if (isError) return <Button title="Retry" onPress={() => refetch()} />;
  return <FlatList data={data} renderItem={({ item }) => <Card p={item} />} />;
}
```

ARCHITECTURE / FLOW

```
Client state            Server state
(you own it)            (backend owns it)
 - UI toggles            - API lists
 - form inputs           - user records
 -> Redux/Zustand        -> React Query
   useState                (cache + refetch)
```

Q27 STATE MANAGEMENT & DATA FETCHING

How do caching, staleTime, refetching, and query invalidation work in React Query?

THEORY

React Query caches each query by its queryKey and serves the cached data instantly while deciding whether to refetch. staleTime controls how long data is considered fresh; while fresh, no background refetch happens, and after it goes stale React Query refetches on triggers like refocus or remount. gcTime (cacheTime) controls how long unused data stays in memory. Invalidation with queryClient.invalidateQueries marks queries stale so they refetch, which is how you refresh data after a change.

STRONG ANSWER

React Query caches API responses using a unique queryKey, so if I revisit a screen, it can show cached data immediately instead of making another API call. staleTime defines how long the cached data is considered fresh. During that time, React Query won't refetch the data. Once the data becomes stale, it can automatically refetch in the background when needed, such as when the app regains focus or the query runs again. When I update data on the server, such as creating or editing an item, I use invalidateQueries() to mark the related query as stale. React Query then refetches the latest data, keeping the UI in sync with the server.

CODE · TypeScript / TSX

```
import { useQuery, useQueryClient } from '@tanstack/react-query';
function useProducts() {
  return useQuery({
    queryKey: ['products'],
    queryFn: getProducts,
    staleTime: 1000 * 60,   // fresh for 60s, no refetch in that window
    gcTime: 1000 * 60 * 5,  // kept 5 min after unused
  });
}
function RefreshButton() {
  const qc = useQueryClient();
  // mark stale -> triggers a refetch of products
  return <Button title="Refresh" onPress={() =>
    qc.invalidateQueries({ queryKey: ['products'] })} />;
}
```

ARCHITECTURE / FLOW

```
fetch -> [fresh] --staleTime--> [stale]
           |                      |
   serve cache, no refetch    refetch on
                              focus/remount/invalidate
unused -> wait gcTime -> removed from cache
```

Q28 STATE MANAGEMENT & DATA FETCHING

How do you perform mutations and implement optimistic updates with React Query?

THEORY

A mutation is any operation that changes server data, like create, update, or delete, handled with useMutation. Optimistic updates mean you update the UI immediately as if the request already succeeded, then roll back if it fails. You do this in onMutate by cancelling in-flight queries, snapshotting the cache, and writing the expected value, then restoring the snapshot in onError. Finally onSettled invalidates the query to sync with the server's real state.

STRONG ANSWER

useMutation is used for operations that change data on the server, such as creating, updating, or deleting items. It provides methods like mutate() along with loading and error states. For a better user experience, I use optimistic updates. Before the server responds, I immediately update the UI so the app feels faster. If the API call fails, I roll back the change. After the request completes, I invalidate the related query so React Query fetches the latest data from the server and keeps the UI in sync.

CODE · TypeScript / TSX

```tsx
const qc = useQueryClient();
const addTodo = useMutation({
  mutationFn: (text: string) => api.post('/todos', { text }),
  onMutate: async (text) => {
    await qc.cancelQueries({ queryKey: ['todos'] });
    const prev = qc.getQueryData<Todo[]>(['todos']);
    qc.setQueryData<Todo[]>(['todos'], (old = []) => [
      ...old, { id: 'temp', text, done: false },
    ]);
    return { prev }; // context for rollback
  },
  onError: (_e, _text, ctx) => qc.setQueryData(['todos'], ctx?.prev),
  onSettled: () => qc.invalidateQueries({ queryKey: ['todos'] }),
});
```

ARCHITECTURE / FLOW

```
onMutate: cancel + snapshot + apply optimistic UI
        |
   send request
     /       \
 success     failure
   |           |
onSettled    onError: restore snapshot
invalidate -> refetch
```

Q29 STATE MANAGEMENT & DATA FETCHING

When would you choose Redux Toolkit versus React Query versus Zustand?

THEORY

These tools solve different problems. React Query is for server state, the async data fetched from APIs that needs caching and refetching. Zustand is a tiny, simple store for client state with minimal boilerplate and selector-based subscriptions. Redux Toolkit is also for client state but shines in large apps that need strict structure, middleware, devtools, and predictable patterns across a big team. In practice many apps pair React Query for server data with Zustand or Redux for client state.

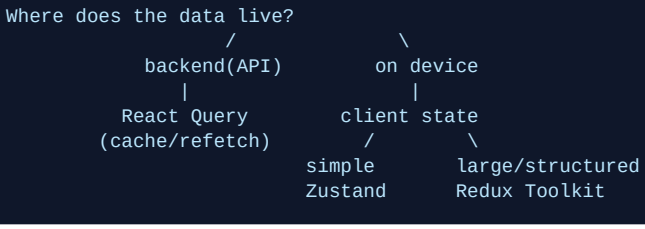
STRONG ANSWER

It depends on the type of state I'm managing. I use React Query for server state, such as data from APIs, because it handles fetching, caching, loading, errors, and refetching automatically. For client state, I choose between Zustand and Redux Toolkit. Zustand is my choice for small to medium projects or simple global state because it's lightweight and easy to use. Redux Toolkit is better for large applications with complex state, where I need a structured architecture, middleware, and powerful debugging tools. In many real-world projects, I use React Query for API data and Zustand or Redux Toolkit for client state, letting each tool do what it's designed for.

CODE · TypeScript / TSX

```tsx
// Server state -> React Query
const { data } = useQuery({ queryKey: ['cart'], queryFn: getCart });
// Client state, lightweight -> Zustand
import { create } from 'zustand';
type UI = { theme: 'light' | 'dark'; toggle: () => void };
export const useUI = create<UI>((set) => ({
  theme: 'light',
  toggle: () => set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })),
}));
const theme = useUI((s) => s.theme); // selector subscription
// Client state, large structured app -> Redux Toolkit
// configureStore + slices + middleware + devtools
```

ARCHITECTURE / FLOW



Q30 STATE MANAGEMENT & DATA FETCHING

What does it mean to normalize state, and how does it help avoid unnecessary re-renders?

THEORY

Normalizing state means storing data in a flat, lookup-friendly shape, typically an object keyed by id plus an array of ids, instead of deeply nested arrays. This avoids duplication, makes updates to a single item cheap, and prevents touching unrelated parts of state. RTK provides createEntityAdapter to do this for you. Combined with memoized selectors, components can subscribe to a single entity by id so only that row re-renders when it changes.

STRONG ANSWER

Normalizing state means storing data in a flat structure instead of deeply nested objects or arrays. Typically, I store items in an object using their id as the key and keep a separate array of IDs. This makes updates more efficient because when one item changes, I only update that item instead of replacing the entire collection. As a result, fewer components re-render, which improves performance. In Redux Toolkit, I can use createEntityAdapter to manage normalized state more easily.

CODE · TypeScript / TSX

```tsx
import { createEntityAdapter, createSlice } from '@reduxjs/toolkit';
const todosAdapter = createEntityAdapter<Todo>();
// state shape: { ids: [...], entities: { [id]: Todo } }
const todosSlice = createSlice({
  name: 'todos',
  initialState: todosAdapter.getInitialState(),
  reducers: {
    addOne: todosAdapter.addOne,
    updateOne: todosAdapter.updateOne, // touches only that entity
  },
});
export const { selectById, selectIds } =
  todosAdapter.getSelectors((s: RootState) => s.todos);
// Row subscribes to ONE entity -> only it re-renders on change
function TodoRow({ id }: { id: string }) {
  const todo = useSelector((s: RootState) => selectById(s, id));
  return <Text>{todo?.text}</Text>;
```

CODE · TypeScript / TSX (continued)

```
}
```

ARCHITECTURE / FLOW

```
Nested (bad):           Normalized (good):
[ {id:1,..},            ids: [1, 2, 3]
  {id:2,..},  -->       entities: {
  {id:3,..} ]             1:{..}, 2:{..}, 3:{..}
                        }
update -> new array     update id=2 -> only row 2
(all rows re-render)    re-renders
4
Animations, Gestures & Performance
Threads, Reanimated worklets, gestures, 60fps, memoization & profiling.
Q31–Q40
4
Animations, Gestures & Performance
Threads, Reanimated worklets, gestures, 60fps, memoization & profiling.
Q31–Q40
4
Animations, Gestures & Performance
Threads, Reanimated worklets, gestures, 60fps, memoization & profiling.
Q31–Q40

4
Animations, Gestures & Performance
Threads, Reanimated worklets, gestures, 60fps, memoization & profiling.
Q31–Q40
```

4 Animations, Gestures & Performance

Threads, Reanimated worklets, gestures, 60fps, memoization & profiling.

Q31–Q40

Q31 ANIMATIONS, GESTURES & PERFORMANCE

What is the difference between the JS thread and the UI thread in React Native, and why do animations sometimes jank?

THEORY

React Native runs your JavaScript logic on the JS thread while the native rendering and gesture work happens on the UI thread (main thread). With the old Animated API, animation values are computed in JS and sent across the bridge, so if the JS thread is busy (data fetching, heavy state updates, big list renders) it can't push updates in time and frames drop. Jank happens because the screen needs a new frame every 16ms but the blocked JS thread misses that window.

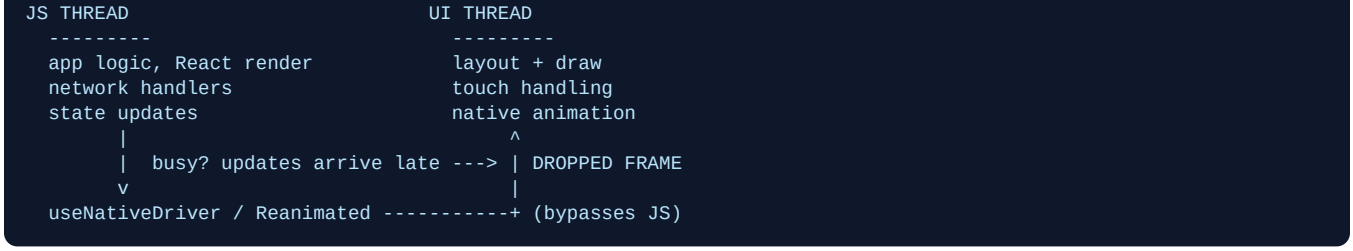
STRONG ANSWER

React Native mainly has two threads: the JavaScript (JS) thread and the UI thread. The JS thread runs my application logic, React components, API calls, and business logic. The UI thread is responsible for rendering the interface, handling touch events, and drawing frames on the screen. Animations can become janky when they're driven by the JS thread and that thread is busy with heavy work, such as large re-renders or expensive calculations. Since the JS thread can't send animation updates fast enough, the animation starts to stutter. To avoid this, I use the native driver when possible or React Native Reanimated, which runs animations directly on the UI thread. This keeps animations smooth even if the JS thread is busy.

CODE · TypeScript / TSX

```tsx
import { Animated, Easing } from 'react-native';
import { useRef, useEffect } from 'react';
export function FadeIn() {
  const opacity = useRef(new Animated.Value(0)).current;
  useEffect(() => {
    Animated.timing(opacity, {
      toValue: 1,
      duration: 300,
      easing: Easing.out(Easing.ease),
      // runs on UI thread, immune to JS thread jank
      useNativeDriver: true,
    }).start();
  }, [opacity]);
  return <Animated.View style={{ opacity }} />;
}
```

ARCHITECTURE / FLOW



Q32 ANIMATIONS, GESTURES & PERFORMANCE

What is the difference between the Animated API and Reanimated, and when would you choose each?

THEORY

The built-in Animated API ships with React Native and works well for simple animations, especially with useNativeDriver: true, but it can only natively drive a limited set of non-layout properties like opacity and transform. Reanimated runs animation logic on the UI thread using worklets, supports layout animations, gestures, and complex interactive choreography with far less jank. For basic fades or slides Animated is fine; for gesture-driven or performance-critical UI, Reanimated is the modern standard.

STRONG ANSWER

The Animated API is built into React Native and is suitable for simple animations like fading, scaling, or moving a component. When used with useNativeDriver, supported animations run on the native side, improving performance. Reanimated is a more powerful animation library. It runs animation logic on the UI thread, making it ideal for complex animations, gestures, and highly interactive user interfaces. It provides features like shared values, animated styles, and smooth gesture-based animations. I use the built-in Animated API for simple animations, and I choose Reanimated when I need high-performance animations, gesture handling, or more advanced animation features.

CODE · TypeScript / TSX

```tsx
// Reanimated equivalent of a fade-in
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withTiming,
} from 'react-native-reanimated';
import { useEffect } from 'react';
export function FadeIn() {
  const opacity = useSharedValue(0);
  useEffect(() => {
    opacity.value = withTiming(1, { duration: 300 });
  }, [opacity]);
  const style = useAnimatedStyle(() => ({ opacity: opacity.value }));
  return <Animated.View style={style} />;
}
```

Q33 ANIMATIONS, GESTURES & PERFORMANCE

What are shared values and useAnimatedStyle in Reanimated, and how do they work together?

THEORY

A shared value (useSharedValue) is a mutable container whose .value can be read and written on both the JS and UI threads, and it is the source of truth for an animation. useAnimatedStyle is a worklet that reads shared values and returns a style object, re-running on the UI thread whenever those values change. Because updates happen on the UI thread, changing a shared value does not trigger a React re-render, which is what keeps animations smooth.

STRONG ANSWER

useSharedValue creates a value that can be updated without causing a React re-render. I use it to store animated values like position, scale, or opacity. useAnimatedStyle uses those shared values to create animated styles. Whenever a shared value changes, the animated style updates automatically on the UI thread, making animations smooth and performant. Together, they let me build high-performance animations because updates happen on the UI thread instead of triggering React re-renders.

CODE · TypeScript / TSX

```tsx
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from 'react-native-reanimated';
import { Button, View } from 'react-native';
export function ScaleBox() {
  const scale = useSharedValue(1);
  const style = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));
  return (
    <View>
      <Animated.View style={[{ width: 80, height: 80 }, style]} />
      <Button title="Grow" onPress={() => (scale.value = withSpring(1.5))} />
    </View>
  );
}
```

ARCHITECTURE / FLOW

```
scale (shared value) ---> useAnimatedStyle (worklet, UI thread)
        |                              |
   .value = withSpring(1.5)            v
        |                     returns { transform:[{scale}] }
        +--- NO React re-render ---> View updates on UI thread
```

Q34 ANIMATIONS, GESTURES & PERFORMANCE

What is a worklet in Reanimated, and what are runOnUI and runOnJS used for?

THEORY

A worklet is a small JavaScript function that Reanimated can serialize and run on the UI thread in a separate JS context, which is how animation logic stays off the main JS thread. runOnUI schedules a function to execute on the UI thread, while runOnJS lets a worklet call back into a regular JS-thread function (for example to update React state or navigate). You need runOnJS because UI-thread worklets can't directly touch React state or most JS-thread APIs.

STRONG ANSWER

A worklet is a function that runs on the UI thread instead of the JavaScript thread. Reanimated uses worklets to execute animation logic directly on the UI thread, which keeps animations smooth even when the JS thread is busy. runOnJS is used when a worklet needs to call JavaScript code, such as updating React state, navigating to another screen, or calling a normal JavaScript function. runOnUI does the opposite—it lets me start a worklet from the JavaScript thread. In short, worklets run on the UI thread, React state runs on the JS thread, runOnJS goes from UI → JS, and runOnUI goes from JS → UI.

CODE · TypeScript / TSX

```tsx
import { Gesture } from 'react-native-gesture-handler';
import { useSharedValue, runOnJS } from 'react-native-reanimated';
import { useState } from 'react';
export function useTapCounter() {
  const [count, setCount] = useState(0);
  const pressed = useSharedValue(false);
  const tap = Gesture.Tap().onEnd(() => {
    'worklet';
    pressed.value = true; // UI-thread safe
    // setCount is a JS-thread fn, must hop back
    runOnJS(setCount)((c: number) => c + 1);
  });
  return { tap, count };
}
```

ARCHITECTURE / FLOW

```
JS THREAD                          UI THREAD
  ---------                          ---------
  React state (setCount)             worklets, shared values
        ^                                  |
        |  runOnJS(setCount) <-------------+  (worklet calls back)
        |                                  ^
        +-------- runOnUI(fn) ------------+  (start a worklet)
```

Q35 ANIMATIONS, GESTURES & PERFORMANCE

How do you handle gestures with react-native-gesture-handler, for example a pan gesture?

THEORY

react-native-gesture-handler replaces React Native's default touch system with native gesture recognizers that run on the UI thread, so gestures stay responsive even when the JS thread is busy. You build gestures with the Gesture API (Gesture.Pan, Gesture.Tap, Gesture.Pinch) and attach them through a GestureDetector. Combined with Reanimated shared values, gesture callbacks are worklets that update animation state directly on the UI thread.

STRONG ANSWER

react-native-gesture-handler provides smooth and reliable gesture handling. For a pan gesture, I create a Gesture.Pan(), handle events like onUpdate and onEnd, and wrap the component with a GestureDetector. Inside the gesture callbacks, I update Reanimated shared values, and useAnimatedStyle uses those values to move the UI. Since the gesture and animation run on the UI thread, the interaction stays smooth even if the JavaScript thread is busy.

CODE · TypeScript / TSX

```tsx
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from 'react-native-reanimated';
export function Draggable() {
  const x = useSharedValue(0);
  const y = useSharedValue(0);
  const pan = Gesture.Pan()
    .onUpdate((e) => {
      x.value = e.translationX;
      y.value = e.translationY;
    })
    .onEnd(() => {
```

CODE · TypeScript / TSX (continued)

```
      x.value = withSpring(0);
      y.value = withSpring(0);
    });
  const style = useAnimatedStyle(() => ({
    transform: [{ translateX: x.value }, { translateY: y.value }],
  }));
  return (
    <GestureDetector gesture={pan}>
      <Animated.View style={[{ width: 100, height: 100 }, style]} />
    </GestureDetector>
  );
}
```

Q36 ANIMATIONS, GESTURES & PERFORMANCE

What does it mean to achieve 60fps in React Native, and what commonly causes dropped frames?

THEORY

Most screens refresh 60 times per second, giving each frame a budget of about 16ms (roughly 8ms on 120Hz displays) to compute and render. If either the JS thread or the UI thread takes longer than that budget, the frame is dropped and the user perceives stutter. Common causes include heavy synchronous work on the JS thread, large or unoptimized list rendering, expensive re-renders, and animating layout properties from JS instead of natively.

STRONG ANSWER

Achieving 60 FPS means the app renders 60 frames every second, which gives users smooth animations and scrolling. Since one frame has about 16 milliseconds to render, any work that takes longer than that can cause dropped frames and make the UI feel laggy. Common causes are heavy JavaScript work, unnecessary re-renders, large lists, expensive layouts, and JavaScript-driven animations. To avoid dropped frames, I optimize re-renders with React.memo, useMemo, and useCallback, use FlatList or FlashList for large lists, and use Reanimated or the native driver so animations run on the UI thread.

CODE · TypeScript / TSX

```
import { InteractionManager } from 'react-native';
import { useEffect, useState } from 'react';
// Defer heavy work until after animations/interactions finish,
// protecting the 16ms-per-frame budget during the transition.
export function useDeferredData(load: () => Promise<unknown>) {
  const [data, setData] = useState<unknown>(null);
  useEffect(() => {
    const task = InteractionManager.runAfterInteractions(async () => {
      const result = await load();
      setData(result);
    });
    return () => task.cancel();
  }, [load]);
  return data;
}
```

ARCHITECTURE / FLOW

```
Frame budget at 60fps = ~16.6ms per frame
  |<------ 16.6ms ------>|<------ 16.6ms ------>|
  [ JS + UI work fits ]   [ work fits ]          OK = smooth
  |<------ 16.6ms ------>|<------ 16.6ms ------>|
  [ work overruns........|.....]  X DROPPED      jank/stutter
```

Q37 ANIMATIONS, GESTURES & PERFORMANCE

How do you prevent unnecessary re-renders in React Native using React.memo, useCallback, and useMemo?

THEORY

React re-renders a component whenever its state or props change, and a parent re-render cascades to children even if their props are identical. React.memo skips re-rendering a child when its props are shallow-equal, useCallback memoizes a function so its reference stays stable between renders, and useMemo caches an expensive computed value. Used together they stop reference churn from triggering wasted renders, which is critical for list items and animation-heavy screens.

STRONG ANSWER

By default, when a parent component re-renders, its child components also re-render. To avoid unnecessary re-renders, I use React.memo, useCallback, and useMemo when they provide a real performance benefit. React.memo prevents a child component from re-rendering if its props haven't changed. useCallback memoizes functions so their reference stays the same when passing them to child components. useMemo memoizes expensive calculations or objects so they aren't recreated on every render. I don't use them everywhere. I use them only when profiling shows unnecessary re-renders, especially in list items or performance-critical screens.

CODE · TypeScript / TSX

```tsx
import React, { useCallback, useMemo, useState } from 'react';
import { FlatList } from 'react-native';
const Row = React.memo(function Row({ item, onPress }: any) {
  return <Item title={item.title} onPress={onPress} />;
});
export function List({ data }: { data: any[] }) {
  const [, force] = useState(0);
  // stable reference -> Row's React.memo stays effective
  const onPress = useCallback((id: string) => force((n) => n + 1), []);
  // cached unless data changes
  const sorted = useMemo(() => [...data].sort(), [data]);
  return (
    <FlatList
      data={sorted}
      renderItem={({ item }) => <Row item={item} onPress={onPress} />}
      keyExtractor={(i) => i.id}
    />
  );
}
```

Q38 ANIMATIONS, GESTURES & PERFORMANCE

How do you tune FlatList performance with props like getItemLayout, keyExtractor, windowSize, and removeClippedSubviews?

THEORY

FlatList is virtualized, rendering only items near the viewport, but it still has to measure, key, and recycle rows efficiently. getItemLayout lets it skip measurement for fixed-height rows so scrolling and scroll-to-index are instant, keyExtractor gives stable identity to avoid remounting, windowSize controls how many screens of content are kept rendered, and removeClippedSubviews detaches off-screen views to save memory. Pairing these with memoized row components keeps long lists smooth.

STRONG ANSWER

FlatList is already optimized with virtualization, but for large lists I tune a few important props. keyExtractor provides a stable key for each item, helping React update the list efficiently. getItemLayout is useful when all items have the same height because it skips measuring and improves scrolling and scrollToIndex performance. windowSize controls how many items are rendered around the visible area, helping balance performance and memory usage. removeClippedSubviews removes off-screen views, reducing memory usage, especially on Android. I also memoize the renderItem component using React.memo and adjust props like initialNumToRender and maxToRenderPerBatch for very large lists.

CODE · TypeScript / TSX

```tsx
import { FlatList } from 'react-native';
import React from 'react';
const ROW_HEIGHT = 72;
const Row = React.memo(({ item }: any) => <Item data={item} />);
export function BigList({ data }: { data: any[] }) {
  return (
    <FlatList
      data={data}
      renderItem={({ item }) => <Row item={item} />}
      keyExtractor={(i) => i.id}
      getItemLayout={(_, index) => ({
        length: ROW_HEIGHT,
        offset: ROW_HEIGHT * index,
        index,
      })}
      windowSize={7}
      initialNumToRender={10}
      maxToRenderPerBatch={10}
      removeClippedSubviews
    />
  );
}
```

ARCHITECTURE / FLOW

```
FlatList window (windowSize controls how many screens kept)
   [ rendered above ]   <- recycled/clipped
  +--------------------+
  |  VISIBLE VIEWPORT  |  <- rendered
  +--------------------+
   [ rendered below ]   <- recycled/clipped
  getItemLayout(index) -> offset = ROW_HEIGHT * index (no measure)
```

Q39 ANIMATIONS, GESTURES & PERFORMANCE

What is the Hermes engine and what does it improve in a React Native app?

THEORY

Hermes is a lightweight JavaScript engine built by Meta specifically for React Native, and it is the default engine on modern versions. It compiles JavaScript to bytecode ahead of time during the build, so the app skips parsing and compiling JS at runtime. This mainly improves startup time (TTI), reduces memory usage, and shrinks app size, with better tooling for profiling and debugging compared to JSC.

STRONG ANSWER

Hermes is the JavaScript engine optimized for React Native and is the default engine in modern React Native apps. Its main goal is to improve app performance. Hermes compiles JavaScript into bytecode during the build process, so the app starts faster because it doesn't need to parse and compile all the JavaScript at launch. It also reduces memory usage and can decrease app size. I use Hermes because it improves startup performance, makes the app more responsive, and works well with React Native debugging and profiling tools.

CODE · TypeScript / TSX

```
// android/gradle.properties
// hermesEnabled=true   (default on modern RN)
// ios/Podfile
// :hermes_enabled => true
// Confirm Hermes is active at runtime:
export function isHermes(): boolean {
  // HermesInternal is injected globally only when Hermes runs
  return !!(global as any).HermesInternal;
}
// Usage
if (isHermes()) {
  console.log('Running on Hermes (bytecode, faster startup)');
}
```

ARCHITECTURE / FLOW

```
WITHOUT Hermes (JSC):
    ship JS source -> device parses + compiles -> runs   (slow start)
  WITH Hermes:
    build: JS -> bytecode  ===>  device loads bytecode -> runs
                                 (faster TTI, less memory)
```

Q40 ANIMATIONS, GESTURES & PERFORMANCE

How do you profile and measure performance in a React Native app?

THEORY

You can't optimize what you don't measure, so the first step is to capture real data. The on-device Perf Monitor shows live JS and UI thread frame rates, React Native DevTools (and the React Profiler) reveal component render counts and timings, and tools like Flipper or Hermes sampling profilers capture CPU traces and bridge traffic. The goal is to find which thread is overloaded and which components or operations are the bottleneck before changing code.

STRONG ANSWER

My approach is to measure first and optimize second. I don't guess where the problem is. I start with the React Native Performance Monitor to check if the JS thread or UI thread is dropping frames. If I suspect unnecessary re-renders, I use React DevTools or the React Profiler to identify which components are rendering too often. For deeper performance analysis, I use Flipper and Hermes profiling tools to inspect CPU usage and overall app performance. Once I identify the bottleneck, I apply the appropriate optimization, such as reducing re-renders, optimizing lists, or moving animations to Reanimated, and then I profile again to verify the improvement.

CODE · TypeScript / TSX

```
import React, { Profiler } from 'react';
// React's built-in Profiler logs render timing programmatically.
function onRender(
  id: string,
  phase: 'mount' | 'update',
  actualDuration: number,
```

CODE · TypeScript / TSX (continued)

```
) {
  if (actualDuration > 16) {
    console.warn(`[perf] ${id} ${phase} took ${actualDuration.toFixed(1)}ms`);
  }
}
export function ProfiledScreen({ children }: { children: React.ReactNode }) {
  return (
    <Profiler id="Screen" onRender={onRender}>
      {children}
    </Profiler>
  );
}
// Also: shake device -> 'Show Perf Monitor' for live JS/UI fps.
```

ARCHITECTURE / FLOW

```
MEASURE  ->  IDENTIFY        ->  FIX            ->  RE-MEASURE
  -------      --------             ---              ----------
  Perf Monitor which thread?       memo / native      confirm fps
  RN DevTools  which component?    driver / list      restored
  Flipper/CPU  what bridge work?   tuning
5
Native Modules & the New Architecture
Bridge vs JSI, TurboModules, Fabric, Codegen and Swift native modules.
Q41–Q50
5
Native Modules & the New Architecture
Bridge vs JSI, TurboModules, Fabric, Codegen and Swift native modules.
Q41–Q50
5
Native Modules & the New Architecture
Bridge vs JSI, TurboModules, Fabric, Codegen and Swift native modules.
Q41–Q50

5
Native Modules & the New Architecture
Bridge vs JSI, TurboModules, Fabric, Codegen and Swift native modules.
Q41–Q50
```

5 Native Modules & the New Architecture

Bridge vs JSI, TurboModules, Fabric, Codegen and Swift native modules.

Q41–Q50

Q41 NATIVE MODULES & THE NEW ARCHITECTURE

What is a native module in React Native, and when would you actually need to write one?

THEORY

A native module is a piece of platform code (Swift, Objective-C, Kotlin, Java) exposed to JavaScript so your JS can call capabilities React Native doesn't ship out of the box. You reach for one when a feature needs a platform API or SDK that has no JS equivalent, when you need performance beyond what JS can give, or when you must reuse existing native code.

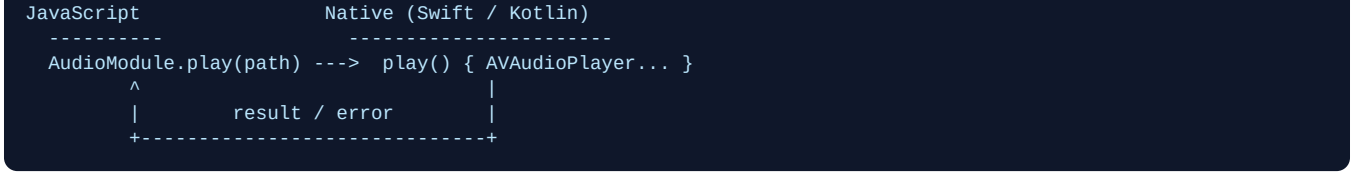
STRONG ANSWER

A native module is a way for React Native's JavaScript code to communicate with native Android or iOS code. I use it when I need access to platform-specific features or native SDKs that aren't available through React Native or an existing library. For example, if I need to integrate a payment SDK, a Bluetooth device, advanced camera features, or a custom native API, I would write a native module. Otherwise, I prefer using existing community libraries because they save development time and are well tested.

CODE · TypeScript

```typescript
// JS side calling a native module
import { NativeModules } from 'react-native';
const { AudioModule } = NativeModules;
// AudioModule is implemented natively in Swift.
// Here we just call it like any JS object.
export async function playClip(path: string) {
  try {
    const durationMs = await AudioModule.play(path);
    console.log('Playing, duration:', durationMs);
  } catch (e) {
    console.warn('Native playback failed', e);
  }
}
```

ARCHITECTURE / FLOW



Q42 NATIVE MODULES & THE NEW ARCHITECTURE

Can you describe the old bridge architecture and what its main bottlenecks were?

THEORY

In the legacy architecture, the JS thread and native threads were fully separate and communicated only through an asynchronous bridge. Every call had to be serialized to JSON, queued, and sent across, which made the bridge a throughput choke point. The two big problems were the serialization cost of converting data to and from JSON, and the forced async nature meaning JS could never call native synchronously.

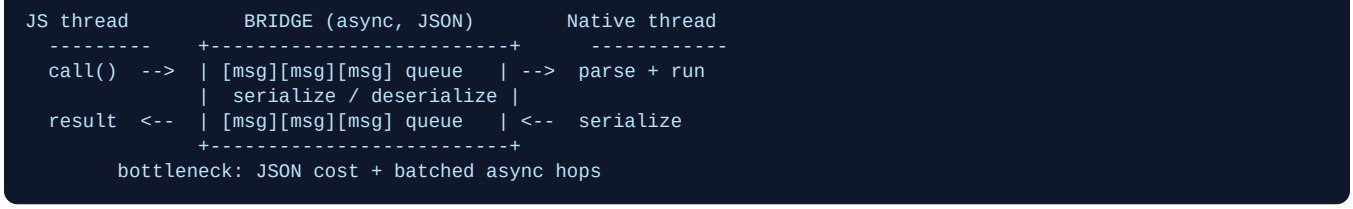
STRONG ANSWER

In the old React Native architecture, the JavaScript thread and the native side were separated and communicated through a bridge. Whenever JavaScript needed to interact with native code, the data had to be serialized, sent across the bridge, and deserialized on the other side. The main problem was performance. Frequent communication, large amounts of data, or JavaScript-driven animations could overload the bridge, causing delays, dropped frames, and janky animations. The new React Native architecture replaces the bridge with JSI, which allows JavaScript to communicate directly with native code without serialization, making interactions faster and more efficient.

CODE · TypeScript

```ts
// Conceptual: every call becomes a serialized message.
// JS -> bridge -> native (and back), all async + JSON.
NativeModules.AudioModule.play('/clip.mp3');
// Internally roughly:
//   1. arguments serialized to JSON
//   2. message pushed onto the bridge queue (batched)
//   3. native thread dequeues, parses JSON
//   4. native runs play(), serializes result
//   5. result travels back async to JS
// You could NOT do this on the old bridge:
// const x = NativeModules.AudioModule.playSync(); // no sync return
```

ARCHITECTURE / FLOW



Q43 NATIVE MODULES & THE NEW ARCHITECTURE

What is JSI (the JavaScript Interface), explained as simply as you can?

THEORY

JSI is a lightweight C++ layer that lets JavaScript hold direct references to native objects and call their methods without serializing to JSON. Instead of message passing, native code installs host objects/functions directly into the JS runtime, so a JS call becomes essentially a regular function call into C++. This enables synchronous calls and removes the bridge as a bottleneck.

STRONG ANSWER

JSI, or JavaScript Interface, is the technology behind React Native's New Architecture. It allows JavaScript to communicate directly with native code without going through the old bridge. In the old architecture, data had to be serialized and sent across the bridge. With JSI, JavaScript can call native functions directly, which makes communication much faster and reduces latency. JSI is the foundation for TurboModules and Fabric, helping React Native deliver better performance and a smoother user experience.

CODE · SWIFT

```
// Conceptual C++ JSI: install a host function into JS runtime.
#include <jsi/jsi.h>
using namespace facebook::jsi;
void install(Runtime& rt) {
  auto multiply = Function::createFromHostFunction(
    rt,
    PropNameID::forAscii(rt, "multiply"),
    2,
    [](Runtime& rt, const Value&, const Value* args, size_t) -> Value {
      double a = args[0].asNumber();
      double b = args[1].asNumber();
      return Value(a * b); // direct return, no JSON
    });
  // Now JS can call global.multiply(6, 7) synchronously.
  rt.global().setProperty(rt, "multiply", multiply);
}
```

ARCHITECTURE / FLOW

```
OLD bridge:  JS --JSON--> queue --JSON--> Native (async)
  JSI:         JS  ===direct C++ call===  Native
               (host objects live inside the JS runtime,
                no serialization, can be synchronous)
```

Q44 NATIVE MODULES & THE NEW ARCHITECTURE

How do TurboModules differ from the old-style native modules?

THEORY

TurboModules are native modules built on JSI instead of the async bridge, so JS calls into them are direct C++ calls with no JSON serialization. They are also lazily loaded, meaning a module is initialized only when JS first uses it, instead of all modules loading at startup. They use Codegen to produce type-safe interfaces from a TypeScript spec.

STRONG ANSWER

TurboModules are the new way of writing native modules in React Native's New Architecture. Compared to the old native modules, they're faster and more efficient because they use JSI instead of the bridge. The old native modules were loaded when the app started, and every call had to go through the bridge. TurboModules are loaded only when they're actually needed (lazy loading), and JavaScript communicates with them directly through JSI. This improves startup time and overall performance. They also support code generation, which provides better type safety between JavaScript and native code.

CODE · TypeScript

```
// NativeAudioModule.ts  (the TurboModule spec)
import type { TurboModule } from 'react-native';
import { TurboModuleRegistry } from 'react-native';
export interface Spec extends TurboModule {
  // Codegen reads these signatures to generate native protocols.
```

CODE · TypeScript (continued)

```
  play(path: string): Promise<number>;
  stop(): void;
  getVolume(): number; // can be synchronous on JSI
}
export default TurboModuleRegistry.getEnforcing<Spec>('AudioModule');
```

ARCHITECTURE / FLOW

```
Old module:  eager load all  +  bridge (JSON, async)
  TurboModule: lazy load on first use + JSI (direct, sync-capable)
  JS  ->  TurboModuleRegistry.getEnforcing('AudioModule')
      ->  JSI host object  ->  native Swift/Kotlin impl
```

Q45 NATIVE MODULES & THE NEW ARCHITECTURE

What is Fabric, and what problem does the new rendering system solve?

THEORY

Fabric is React Native's new rendering system, also built on JSI, that replaces the old UIManager bridge for creating and updating native views. It builds an immutable shadow tree in C++ shared across threads and can apply layout and view updates synchronously, which improves consistency and responsiveness. It also enables better interop, concurrent React features, and reduced UI jank.

STRONG ANSWER

Fabric is React Native's new rendering system and part of the New Architecture. Its job is to convert React components into native UI elements like UIView on iOS or android.view on Android. The old rendering system relied on the bridge, which could introduce delays and dropped frames. Fabric uses JSI for more direct communication with native code, making rendering faster and more efficient. It also works better with modern React features like concurrent rendering and Suspense. As a result, Fabric provides smoother UI updates, better performance, and a more responsive user experience.

CODE · TypeScript / TSX

```
// You usually enable Fabric via the New Architecture flag,
// not by writing Fabric code directly.
// iOS: ios/Podfile.properties.json
{
  "newArchEnabled": "true"
}
// Android: android/gradle.properties
// newArchEnabled=true
// With Fabric on, your normal JSX renders through the new
// C++ shadow tree + synchronous commit pipeline automatically.
export function Card() {
  return <View style={{ padding: 16 }}><Text>Hello</Text></View>;
}
```

ARCHITECTURE / FLOW

```
React tree (JS)
        |  JSI
        v
  Shadow tree (C++, immutable, shared)
        |  commit (can be synchronous)
        v
  Mounting layer -> native UIView / android.view.View
  Old UIManager: async bridge ops -> tearing / dropped frames
```

Q46 NATIVE MODULES & THE NEW ARCHITECTURE

What is Codegen, and how does it give you type-safe native specs?

THEORY

Codegen is a build-time tool that reads your TypeScript spec files for TurboModules and Fabric components and generates the corresponding native interfaces (Objective-C++/Java/C++). This guarantees the JS contract and the native implementation stay in sync, since the native side must conform to generated protocols. It runs automatically during the build, so type mismatches surface as compile errors rather than runtime crashes.

STRONG ANSWER

Codegen is a tool in React Native's New Architecture that automatically generates native code from a TypeScript specification. I define a TypeScript interface describing the methods, parameters, and return types of a native module or component. Codegen then generates the corresponding native code for iOS and Android. This keeps the JavaScript and native sides in sync and provides type safety. If the interface changes and the native implementation isn't updated, the build fails instead of causing runtime errors.

CODE · TypeScript

```
// package.json — tells Codegen where the specs live.
{
  "codegenConfig": {
    "name": "AudioModuleSpec",
    "type": "modules",
    "jsSrcsDir": "src/specs",
    "ios": { "componentProvider": {} }
  }
}
// Codegen reads src/specs/NativeAudioModule.ts and emits, e.g.:
//   - iOS:    @protocol NativeAudioModuleSpec  (Obj-C++)
//   - Android: abstract class NativeAudioModuleSpec (Java)
// Your Swift/Kotlin code must conform, or the build fails.
```

ARCHITECTURE / FLOW

```
NativeAudioModule.ts (TS spec)
            |
        Codegen (build time)
       /            \
  iOS protocol     Android abstract class
  (Obj-C++)        (Java/Kotlin)
       \            /
   native impl must conform -> mismatch = compile error
```

Q47 NATIVE MODULES & THE NEW ARCHITECTURE

How would you write a simple native module in Swift that exposes a method to JavaScript?

THEORY

On iOS you create a Swift class subclassing NSObject and expose it with the @objc(Name) annotation, plus a small Objective-C bridging macro (RCT_EXTERN_MODULE) so React Native can find it. Methods are exposed with RCT_EXTERN_METHOD, and you typically return results via a resolver/rejecter (promise) or a callback. The Swift method names and parameter labels must match the macro declarations exactly.

STRONG ANSWER

To create a native module in Swift, I create a class that extends NSObject and expose it to React Native using @objc. I then expose the methods I want JavaScript to call. React Native registers the module, allowing JavaScript to access it. On the JavaScript side, I import the module from NativeModules and call its methods just like any other JavaScript function. If the operation is asynchronous, I expose it as a Promise so it can be used with async/await. In practice, I only write native modules when I need to integrate a native SDK or access platform-specific features that aren't available through existing React Native libraries.

CODE · SWIFT

```swift
// AudioModule.swift
import Foundation
@objc(AudioModule)
class AudioModule: NSObject {
  @objc(play:resolver:rejecter:)
  func play(_ path: String,
            resolver resolve: @escaping RCTPromiseResolveBlock,
            rejecter reject: @escaping RCTPromiseRejectBlock) {
    guard let url = URL(string: path) else {
      reject("bad_path", "Invalid path", nil); return
    }
    // ... start AVAudioPlayer with url ...
    resolve(1200) // duration in ms back to JS
  }
  @objc static func requiresMainQueueSetup() -> Bool { false }
}
// AudioModule.m (Obj-C bridge)
// #import <React/RCTBridgeModule.h>
// @interface RCT_EXTERN_MODULE(AudioModule, NSObject)
// RCT_EXTERN_METHOD(play:(NSString *)path
//   resolver:(RCTPromiseResolveBlock)resolve
//   rejecter:(RCTPromiseRejectBlock)reject)
// @end
```

Q48 NATIVE MODULES & THE NEW ARCHITECTURE

When and how do you wrap a native view, and what is a ViewManager?

THEORY

When you need to render an actual platform UI control (a map, a video surface, a camera preview) you wrap it with a native UI component rather than a plain native module. On the old architecture this is a ViewManager (RCTViewManager) that creates the view and exposes props; on Fabric it's a codegen'd native component from a spec. You expose props from native to JS and emit events back up.

STRONG ANSWER

I write a native module when I only need native functionality, but if I need to display a native UI component, I wrap it as a native view using a ViewManager. A ViewManager creates and manages the native view, exposes its properties to JavaScript, and sends native events back to JavaScript. This lets React Native use native UI components just like built-in components. Examples include maps, video players, camera previews, or other platform-specific UI components. In the New Architecture, this is handled through Fabric and Codegen.

CODE · SWIFT

```swift
// AudioWaveformViewManager.swift
import UIKit
@objc(AudioWaveformViewManager)
class AudioWaveformViewManager: RCTViewManager {
```

CODE · SWIFT (continued)

```
  override func view() -> UIView! {
    return WaveformView() // your custom UIView subclass
  }
  override static func requiresMainQueueSetup() -> Bool { true }
}
// AudioWaveformViewManager.m
// @interface RCT_EXTERN_MODULE(AudioWaveformViewManager, RCTViewManager)
// RCT_EXPORT_VIEW_PROPERTY(color, NSString)
// RCT_EXPORT_VIEW_PROPERTY(onTap, RCTBubblingEventBlock)
// @end
// JS usage:
// const Waveform = requireNativeComponent('AudioWaveformView');
// <Waveform color="#0af" onTap={e => ...} />
```

ARCHITECTURE / FLOW

```
<Waveform color onTap />        (JS component)
          |  props down / events up
          v
  ViewManager (creates + configures the native view)
          |
          v
  WaveformView : UIView   (the real native control on screen)
```

Q49 NATIVE MODULES & THE NEW ARCHITECTURE

What is autolinking, and how does a native module actually get connected to your app?

THEORY

Autolinking is the mechanism that automatically discovers native modules in your dependencies and wires them into the iOS and Android builds, so you don't manually edit project files. It reads each package's react-native.config.js and metadata, then registers the module during pod install (iOS) and Gradle sync (Android). Before autolinking, you had to manually link libraries, which was error-prone.

STRONG ANSWER

Autolinking is a React Native feature that automatically connects native libraries to your iOS and Android projects. Before autolinking, developers had to manually update Xcode and Gradle files whenever they installed a native package. Now, after installing a library and running pod install on iOS or rebuilding the app, React Native detects the native module and links it automatically. Once linked, the module is available in JavaScript through NativeModules or the library's API. One important thing to remember is that adding a native module requires a native rebuild—it won't work with just a Metro reload.

CODE · TypeScript

```
// react-native.config.js for a local native module
module.exports = {
  dependencies: {
    'audio-module': {
      root: __dirname + '/native/audio-module',
      platforms: {
        ios: { podspecPath: './native/audio-module/AudioModule.podspec' },
        android: { sourceDir: './native/audio-module/android' },
      },
    },
  },
};
// Then:
//   cd ios && pod install     # iOS autolink
//   npx react-native run-android  # Android autolink via Gradle
```

CODE · TypeScript (continued)

```
// After a native rebuild, NativeModules.AudioModule is available.
```

ARCHITECTURE / FLOW

```
node_modules / local pkgs
        |
  RN CLI scans + reads react-native.config.js
        |
   +----+----+
   v         v
 pod install  Gradle sync
 (iOS)        (Android)
        |
  module registered -> available in NativeModules
```

Q50 NATIVE MODULES & THE NEW ARCHITECTURE

How does data and threading work between JS and native, and when do you use callbacks vs promises vs events?

THEORY

Native module methods don't run on the JS thread; they typically run on a native module queue (or a queue you specify), so heavy work won't block JS. You return results to JS in three ways: callbacks (a one-time function, can fire once), promises (resolver/rejecter, ideal for async request/response), and events (an emitter, for streaming or multiple values over time). Only serializable values cross the boundary.

STRONG ANSWER

JavaScript and native code run on different threads and communicate with each other. Native code performs its work and then sends the result back to JavaScript. For communication, I choose the approach based on the use case: Promises for a single asynchronous result, like fetching data or starting audio playback. Callbacks for simple one-time responses, though they're less common today. Events when native needs to continuously send updates, such as download progress, location updates, or audio playback progress. In general, I prefer Promises for one-time operations and Events for ongoing updates.

CODE · TypeScript

```
// Native (Swift) emitting events for streaming updates
@objc(AudioModule)
class AudioModule: RCTEventEmitter {
  override func supportedEvents() -> [String]! { ["onProgress"] }
  func tick(_ ms: Int) {
    sendEvent(withName: "onProgress", body: ["positionMs": ms])
  }
}
// JS: promise for request/response, events for streams
import { NativeEventEmitter, NativeModules } from 'react-native';
const { AudioModule } = NativeModules;
const duration = await AudioModule.play('/clip.mp3'); // promise
const emitter = new NativeEventEmitter(AudioModule);
const sub = emitter.addListener('onProgress', e =>
  console.log('pos', e.positionMs));              // events
// sub.remove() when done
```

ARCHITECTURE / FLOW

```
Native runs OFF the JS thread (module/background queue)
  Callback : native --(fires once)--> cb(result)
  Promise  : await native()  -> resolve(value) / reject(err)
  Event    : native --emit--> emit--> emit--> JS listener (stream)
  Only serializable data crosses: numbers, strings, arrays, objects
6
Offline-First, SQLite, Geolocation & Background Tasks
SQLite, outbox/sync, background geolocation, backoff and idempotency.
Q51–Q60
6
Offline-First, SQLite, Geolocation & Background Tasks
SQLite, outbox/sync, background geolocation, backoff and idempotency.
Q51–Q60
6
Offline-First, SQLite, Geolocation & Background Tasks
SQLite, outbox/sync, background geolocation, backoff and idempotency.
Q51–Q60

6
Offline-First, SQLite, Geolocation & Background Tasks
SQLite, outbox/sync, background geolocation, backoff and idempotency.
Q51–Q60
```

6 Offline-First, SQLite, Geolocation & Background Tasks

SQLite, outbox/sync, background geolocation, backoff and idempotency.

Q51–Q60

Q51 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

What does 'offline-first' mean and why does it matter for a mobile app?

THEORY

Offline-first means the app treats the local device as the source of truth for the UI, so reads and writes succeed even with no network, and the server sync happens in the background. The network is treated as an enhancement, not a requirement, which is critical for mobile where connectivity is unreliable.

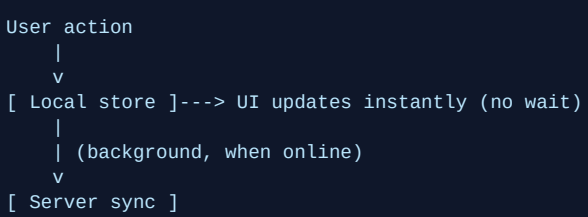
STRONG ANSWER

Offline-first means the app continues to work even when there's no internet connection. Instead of depending on the network, it stores data locally first and syncs it with the server when the connection is available again. This improves the user experience because the app remains fast and responsive, and users don't lose their work due to poor connectivity. It's especially useful for apps like messaging, field service, delivery, or location tracking.

CODE · TypeScript

```typescript
// Offline-first write: succeed locally, sync later
async function recordLocation(point: LocationPoint) {
  // 1) Always write to the local store first (source of truth)
  await db.execute(
    'INSERT INTO locations (lat, lng, ts, synced) VALUES (?, ?, ?, 0)',
    [point.lat, point.lng, point.ts]
  );
  // 2) UI updates immediately from local data
  emit('location-saved', point);
  // 3) Try to sync, but never block the user on it
  trySyncInBackground().catch(() => {
    /* stays buffered; retried later */
  });
}
```

ARCHITECTURE / FLOW



Q52 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you choose between AsyncStorage, MMKV, and SQLite for local storage?

THEORY

AsyncStorage is a simple async key-value store, fine for small flags and settings but slow and unstructured. MMKV is a synchronous, very fast key-value store backed by memory-mapped files, great for frequently read small data. SQLite is a relational database for structured, queryable, large datasets where you need indexes, transactions, and complex queries.

STRONG ANSWER

I choose the storage solution based on the type and amount of data. For simple key-value data like user preferences, tokens, or app settings, I use AsyncStorage or MMKV. If performance is important, I prefer MMKV because it's much faster. For structured or large amounts of data, such as offline records, chat messages, or location history, I use SQLite because it supports tables, queries, indexes, and transactions.

CODE · TypeScript

```typescript
// Rule of thumb in code comments
// AsyncStorage: small, async, simple flags
await AsyncStorage.setItem('onboarded', 'true');
// MMKV: small, synchronous, hot-path reads
import { MMKV } from 'react-native-mmkv';
const storage = new MMKV();
storage.set('lastScreen', 'Map'); // sync, no await
const screen = storage.getString('lastScreen');
// SQLite: structured, large, queryable, transactional
await db.execute(
  'SELECT * FROM locations WHERE synced = 0 ORDER BY ts ASC LIMIT 100'
);
```

ARCHITECTURE / FLOW

```
Small + simple .......... AsyncStorage / MMKV
Small + hot path (sync) . MMKV
Structured + large ...... SQLite
Needs queries/indexes ... SQLite
```

Q53 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you perform basic CRUD operations with SQLite in React Native?

THEORY

Libraries like react-native-quick-sqlite or op-sqlite give you a database handle on which you run parameterized SQL. You always use parameter placeholders (?) instead of string concatenation to avoid SQL injection and quoting bugs, and wrap multi-row work in transactions for speed and atomicity.

STRONG ANSWER

To use SQLite in React Native, I first open the database and create the required tables if they don't already exist. Then I perform CRUD operations using SQL queries. INSERT adds new records, SELECT reads data, UPDATE modifies existing records, and DELETE removes records. I always use parameterized queries to avoid SQL injection and use transactions when inserting or updating multiple records for better performance.

CODE · TypeScript

```
import { open } from '@op-engineering/op-sqlite';
const db = open({ name: 'tracker.db' });
await db.execute(`CREATE TABLE IF NOT EXISTS locations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  lat REAL, lng REAL, ts INTEGER, synced INTEGER DEFAULT 0
)`);
// CREATE
await db.execute('INSERT INTO locations (lat,lng,ts) VALUES (?,?,?)',
  [12.97, 77.59, Date.now()]);
// READ
const { rows } = await db.execute(
  'SELECT * FROM locations WHERE synced = 0');
// UPDATE
await db.execute('UPDATE locations SET synced = 1 WHERE id = ?', [42]);
// DELETE
await db.execute('DELETE FROM locations WHERE id = ?', [42]);
```

Q54 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How would you design an offline write queue (outbox pattern) for pending changes?

THEORY

The outbox pattern stores each pending mutation as a durable row in a local 'outbox' table instead of firing it at the server directly. A separate flush worker reads pending rows in order, sends them, and marks or deletes them on success, so writes survive app restarts and network outages.

STRONG ANSWER

I would use an outbox pattern where every offline action is first saved locally instead of being sent directly to the server. When the device comes back online, a background sync process reads the pending items one by one, sends them to the server, and marks them as completed or removes them from the queue. This makes the app reliable because users can continue working offline, no data is lost if the app closes, and failed requests can be retried automatically.

CODE · TypeScript

```
// Enqueue any mutation durably
async function enqueue(type: string, payload: object) {
  await db.execute(
    `INSERT INTO outbox (type, payload, status, attempts)
     VALUES (?, ?, 'pending', 0)`,
    [type, JSON.stringify(payload)]
  );
}
// Drain in order
async function flushOutbox() {
  const { rows } = await db.execute(
    `SELECT * FROM outbox WHERE status='pending' ORDER BY id ASC`);
  for (const row of rows) {
    try {
      await api.send(row.type, JSON.parse(row.payload));
      await db.execute('DELETE FROM outbox WHERE id=?', [row.id]);
    } catch {
      await db.execute(
        'UPDATE outbox SET attempts = attempts + 1 WHERE id=?', [row.id]);
    }
  }
}
```

ARCHITECTURE / FLOW

```
Mutation --> [ outbox table (durable) ]
                    |  FIFO
                    v
             [ flush worker ] --success--> delete row
                    |
                 failure --> attempts++ (retry later)
```

Q55 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

What sync strategy and conflict resolution would you use between device and server?

THEORY

When the same record changes in two places you need a rule to decide the winner. Last-write-wins (LWW) keeps whichever update has the newest timestamp; it is simple but can silently drop changes. Versioning (a version number or vector clock per record) detects true conflicts so you can merge or prompt instead of blindly overwriting.

STRONG ANSWER

The sync strategy depends on the type of data. For append-only data, like location history or chat messages, I simply sync new records because they don't overwrite existing data. For editable data, I usually use a Last-Write-Wins (LWW) strategy, where the latest update from the server is treated as the source of truth. For more critical data, I use versioning or timestamps so the server can detect conflicts and ask the client to fetch the latest data before updating. The goal is to keep the device and server consistent while preventing data loss.

CODE · TypeScript

```typescript
// Versioned update: server rejects stale writes
async function pushUpdate(record: Rec) {
  const res = await api.put(`/items/${record.id}`, {
    ...record,
    version: record.version, // version client last saw
  });
  if (res.status === 409) {
    // conflict: server has a newer version
    const fresh = await api.get(`/items/${record.id}`);
    const merged = merge(record, fresh); // or last-write-wins
    return pushUpdate({ ...merged, version: fresh.version });
  }
  return res.data; // accepted; version bumped server-side
}
```

ARCHITECTURE / FLOW

```
Client v3 --update(base=v3)--> Server
                                 |
            server still v3? --> accept, bump to v4
            server now v5?  --> 409 conflict
                                 |
                          client re-fetch + merge
```

Q56 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you capture device location in the background while the app is not in the foreground?

THEORY

You can't rely on JS timers in the background; you use a native background-location module such as react-native-background-geolocation or a foreground service. These register an OS-level listener that wakes your handler on location changes or distance/time intervals, and the captured point is written to local storage immediately.

STRONG ANSWER

For background location tracking, I use a dedicated background geolocation library that works with native Android and iOS services. This allows the app to continue receiving location updates even when it's in the background. Whenever a new location is received, I save it immediately to local storage, such as SQLite, and then sync it with the server when the device has network connectivity. I also configure update intervals and distance filters to balance location accuracy with battery usage.

CODE · TypeScript

```
import BackgroundGeolocation from 'react-native-background-geolocation';
BackgroundGeolocation.onLocation(async (location) => {
  // Persist immediately; survives backgrounding/kill
  await db.execute(
    'INSERT INTO locations (lat,lng,ts,synced) VALUES (?,?,?,0)',
    [location.coords.latitude, location.coords.longitude, Date.now()]
  );
});
BackgroundGeolocation.ready({
  desiredAccuracy: BackgroundGeolocation.DESIRED_ACCURACY_HIGH,
  distanceFilter: 25,          // meters between updates
  stopOnTerminate: false,      // keep tracking after app close
  startOnBoot: true,           // resume after device reboot
}).then(() => BackgroundGeolocation.start());
```

ARCHITECTURE / FLOW

```
OS location service (native)
        | onLocation callback
        v
  [ insert into SQLite (synced=0) ]
        |
        v (later, when online)
  [ background sync to server ]
```

Q57 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

What OS-level constraints affect background tasks on iOS and Android, and how do you work with them?

THEORY

Mobile OSes aggressively limit background execution to save battery. iOS gives short background-fetch windows on its own schedule and can suspend or terminate apps, so you can't guarantee periodic timers. Android uses Doze mode and background limits, and long-running work usually requires a foreground service with a persistent notification or WorkManager-style scheduling.

STRONG ANSWER

Background tasks are controlled by the operating system, so an app can't assume it will always run in the background. On iOS, background execution is limited, and the system decides when background tasks can run. On Android, long-running background work, like continuous location tracking, typically requires a foreground service with a persistent notification. To work within these constraints, I save important data locally as soon as it's available and make background sync resumable. Whenever the app gets a chance to run—whether in the background or when it returns to the foreground—I sync any pending data with the server.

CODE · TypeScript

```
import BackgroundFetch from 'react-native-background-fetch';
// iOS/Android: opportunistic windows, not exact timers
BackgroundFetch.configure(
  { minimumFetchInterval: 15, stopOnTerminate: false, startOnBoot: true },
  async (taskId) => {
    // Keep work short and idempotent; OS may cut us off
    await flushOutbox();
    BackgroundFetch.finish(taskId); // MUST signal completion
  },
  (taskId) => {
    BackgroundFetch.finish(taskId); // timeout: finish gracefully
  }
);
```

ARCHITECTURE / FLOW

```
iOS:  app suspended <----> short fetch window (OS-decided)
Android: Doze / limits --> need foreground service + notif
Strategy: persist locally now, flush on ANY wake opportunity
```

Q58 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you implement retry with exponential backoff when syncing through network outages?

THEORY

Retrying immediately and forever hammers the server and drains the battery, so you back off exponentially: wait roughly base * 2^attempt between tries, up to a max cap. Adding jitter (a random offset) prevents many devices from retrying in lockstep (the thundering-herd problem).

STRONG ANSWER

When syncing fails because of a network issue, I don't retry immediately. Instead, I use exponential backoff, where the delay between retries increases after each failure. This reduces unnecessary requests and gives the network or server time to recover. I also add a small random delay, called jitter, so multiple devices don't retry at the same time. If the device is offline, I keep the data safely in local storage and retry when the network becomes available.

CODE · TypeScript

```
async function flushWithBackoff(maxAttempts = 6) {
  for (let attempt = 0; attempt < maxAttempts; attempt++) {
    try {
      await flushOutbox();
      return; // success
    } catch {
      const base = 1000;                // 1s
      const cap = 60_000;               // 60s ceiling
      const delay = Math.min(cap, base * 2 ** attempt);
```

CODE · TypeScript (continued)

```
      const jitter = Math.random() * delay * 0.3;
      await new Promise((r) => setTimeout(r, delay + jitter));
    }
  }
  // give up for now; data stays buffered for next trigger
}
```

ARCHITECTURE / FLOW

```
attempt:  0    1    2    3     4
delay:    1s   2s   4s   8s   16s ... (cap 60s) + jitter
buffer stays in SQLite the whole time -> no data loss
```

Q59 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you detect connectivity changes with NetInfo and react to them?

THEORY

@react-native-community/netinfo reports network state and exposes a subscription that fires on changes. You should check isConnected and isInternetReachable, because a device can be on Wi-Fi yet have no real internet (a captive portal), and trigger a sync on the offline-to-online transition.

STRONG ANSWER

I use @react-native-community/netinfo to monitor the device's network status. I subscribe to connectivity changes, and when the device comes back online, I automatically start syncing any pending data. I check both isConnected and isInternetReachable because being connected to Wi-Fi or mobile data doesn't always mean the internet is actually accessible. To avoid unnecessary sync attempts on unstable networks, I debounce or limit repeated sync triggers.

CODE · TypeScript

```
import NetInfo from '@react-native-community/netinfo';
let wasOnline = false;
const unsubscribe = NetInfo.addEventListener((state) => {
  const online = !!state.isConnected && state.isInternetReachable !== false;
  if (online && !wasOnline) {
    // offline -> online transition: drain the buffer
    flushWithBackoff().catch(() => {});
  }
  wasOnline = online;
});
// On unmount
// unsubscribe();
```

ARCHITECTURE / FLOW

```
NetInfo events:  offline ... offline ... ONLINE
                                          |
                          transition detected (edge)
                                          v
                                  flush the outbox
```

Q60 OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you ensure data integrity (idempotency and dedup) when flushing the queue to the server?

THEORY

A flush can succeed on the server but fail before the client records it, so the client retries and the server sees the same write twice. Idempotency means a repeated request has no extra effect, usually achieved with a client-generated unique id that the server dedupes on, so retries are safe and no duplicate rows appear.

STRONG ANSWER

To ensure data integrity, I make every offline record have a unique client-generated ID, such as a UUID. When the app syncs, the server uses that ID to identify duplicate requests. If the same request is received again, the server ignores it or updates the existing record instead of creating a duplicate. On the client, I only remove an item from the offline queue after the server confirms it was processed successfully. This ensures data isn't lost if the network fails during syncing.

CODE · TypeScript

```typescript
import 'react-native-get-random-values';
import { v4 as uuid } from 'uuid';
// Assign a stable id at capture time
async function capture(p: LocationPoint) {
  await db.execute(
    'INSERT INTO locations (client_id, lat, lng, ts) VALUES (?,?,?,?)',
    [uuid(), p.lat, p.lng, p.ts]
  );
}
// Send the client_id so the server can dedupe (idempotent)
async function syncBatch(rows: Row[]) {
  await api.post('/locations/batch', {
    points: rows.map((r) => ({ id: r.client_id, lat: r.lat, lng: r.lng })),
  }); // server upserts by id -> retry-safe
  const ids = rows.map((r) => r.id);
  await db.execute(
    `DELETE FROM locations WHERE id IN (${ids.map(() => '?').join(',')})`,
    ids); // delete only after confirmed success
}
```

ARCHITECTURE / FLOW

```
client_id=abc --send--> server stores abc
        |  (response lost)
        v
client retries client_id=abc --> server sees abc -> ignores
        => exactly one record, no duplicates
7
Real-Time: WebSockets, WebRTC & CRDTs
WebSockets, reconnection, Yjs CRDTs, presence, WebRTC and scaling.
Q61–Q70
7
Real-Time: WebSockets, WebRTC & CRDTs
WebSockets, reconnection, Yjs CRDTs, presence, WebRTC and scaling.
Q61–Q70
7
Real-Time: WebSockets, WebRTC & CRDTs
WebSockets, reconnection, Yjs CRDTs, presence, WebRTC and scaling.
Q61–Q70

7
Real-Time: WebSockets, WebRTC & CRDTs
WebSockets, reconnection, Yjs CRDTs, presence, WebRTC and scaling.
Q61–Q70
```

7 Real-Time: WebSockets, WebRTC & CRDTs

WebSockets, reconnection, Yjs CRDTs, presence, WebRTC and scaling.

Q61–Q70

Q61 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

When would you choose HTTP polling, long-polling, SSE, or WebSockets for a real-time feature?

THEORY

Polling repeatedly asks the server on an interval, which is simple but wasteful and laggy. Long-polling holds the request open until data is ready, reducing empty responses but still one message per round-trip. SSE (Server-Sent Events) is a one-way server-to-client stream over HTTP, great for feeds and notifications. WebSockets give a single persistent full-duplex connection, which is the right fit for chat, collaboration, and anything bidirectional and high-frequency.

STRONG ANSWER

I choose the technology based on how real-time the feature needs to be and whether communication is one-way or two-way. HTTP Polling is good for data that changes occasionally, like refreshing a dashboard every minute. Long Polling is useful when the server should respond only when new data is available, reducing unnecessary requests. Server-Sent Events (SSE) are ideal when only the server needs to continuously push updates to the client, such as live notifications or stock prices. WebSockets are my choice for true real-time, two-way communication, like chat, typing indicators, video calls, multiplayer games, or collaborative editing.

CODE · TypeScript

```typescript
// Quick decision sketch in code comments
// 1) Polling: simple status checks
setInterval(() => fetch('/api/status').then(r => r.json()), 30000);
// 2) SSE: one-way server push (notifications, feeds)
const es = new EventSource('/api/notifications');
es.onmessage = (e) => console.log('push:', e.data);
// 3) WebSocket: bidirectional, high-frequency (chat, collab)
const ws = new WebSocket('wss://api.example.com/collab');
ws.onmessage = (e) => console.log('msg:', e.data);
ws.onopen = () => ws.send(JSON.stringify({ type: 'join', room: 'doc-1' }));
```

ARCHITECTURE / FLOW

```
Direction & frequency -> transport
  one-shot/low freq  ........ Polling
  delayed pull       ........ Long-polling
  server -> client   ........ SSE
  client <-> server  ........ WebSocket  (chat, collab)
  Polling:    C --req--> S   C --req--> S   (wasteful)
  WebSocket:  C <=======> S   (one persistent full-duplex pipe)
```

Q62 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How does a WebSocket connection actually work, from handshake to full-duplex messaging?

THEORY

A WebSocket starts as a normal HTTP GET with an `Upgrade: websocket` header and a random `Sec-WebSocket-Key`. The server replies with 101 Switching Protocols and a hashed `Sec-WebSocket-Accept`, after which the same TCP socket is reused for the WebSocket protocol. From then on both sides send frames independently in either direction, which is what makes it full-duplex. The connection stays open until either side sends a close frame.

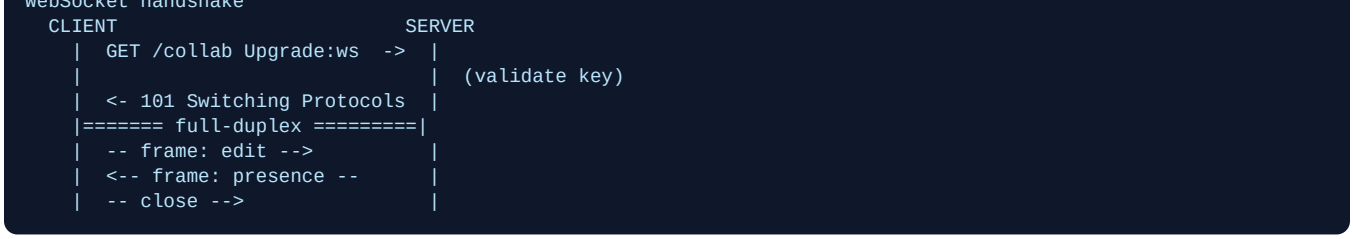
STRONG ANSWER

A WebSocket connection starts as a normal HTTP request. The client asks the server to upgrade the connection to WebSocket. If the server accepts, it responds with a 101 Switching Protocols status, and the connection stays open. After that, both the client and server can send messages to each other at any time without creating new HTTP requests. This full-duplex communication makes WebSockets ideal for real-time features like chat, live collaboration, typing indicators, and video calls.

CODE · TypeScript

```typescript
// Conceptual handshake (the browser/RN does this for you)
// Client request:
//   GET /collab HTTP/1.1
//   Upgrade: websocket
//   Connection: Upgrade
//   Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
//   Sec-WebSocket-Version: 13
//
// Server response:
//   HTTP/1.1 101 Switching Protocols
//   Upgrade: websocket
//   Connection: Upgrade
//   Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
const ws = new WebSocket('wss://api.example.com/collab');
ws.onopen = () => console.log('upgraded -> full-duplex open');
ws.onclose = (e) => console.log('closed', e.code, e.reason);
```

ARCHITECTURE / FLOW



Q63 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How do you set up a WebSocket connection in React Native, including sending messages and cleaning up?

THEORY

React Native ships a global WebSocket implementation, so the API matches the browser. The pattern is to open the socket inside an effect, attach `onopen`, `onmessage`, `onerror`, and `onclose` handlers, and then close it in the cleanup function. Forgetting cleanup is a common leak that leaves duplicate sockets open across re-renders or screen remounts. You also generally want the socket in a ref so re-renders don't recreate it.

STRONG ANSWER

React Native has a built-in WebSocket API, so the setup is similar to the web. I create the WebSocket connection inside useEffect, listen for events like onopen, onmessage, onerror, and onclose, and store the socket in a useRef so it persists across re-renders. To send a message, I first check that the connection is open using readyState, then call send(). Finally, I close the socket in the useEffect cleanup function to avoid memory leaks or multiple active connections when the user leaves the screen.

CODE · TypeScript

```
import { useEffect, useRef, useCallback } from 'react';
function useCollabSocket(room: string) {
  const wsRef = useRef<WebSocket | null>(null);
  useEffect(() => {
    const ws = new WebSocket(`wss://api.example.com/collab/${room}`);
    wsRef.current = ws;
    ws.onopen = () => ws.send(JSON.stringify({ type: 'join', room }));
    ws.onmessage = (e) => handleMessage(JSON.parse(e.data));
    ws.onerror = (e) => console.warn('ws error', e);
    ws.onclose = () => console.log('ws closed');
    return () => ws.close(1000, 'unmount'); // cleanup
  }, [room]);
  const send = useCallback((msg: object) => {
    const ws = wsRef.current;
    if (ws?.readyState === WebSocket.OPEN) ws.send(JSON.stringify(msg));
  }, []);
  return { send };
}
```

Q64 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

What reconnection strategy and heartbeat mechanism do you use to keep a WebSocket healthy on mobile?

THEORY

Mobile connections drop constantly when the app backgrounds, switches networks, or hits flaky signal. The standard fix is exponential backoff with jitter so reconnect attempts don't hammer the server or sync into a thundering herd. A heartbeat (ping/pong) detects dead connections that TCP still thinks are alive, since a silent half-open socket can otherwise hang for minutes. On reconnect you typically re-join rooms and re-sync state.

STRONG ANSWER

On mobile, network connections can drop frequently, so I always implement automatic reconnection. If the WebSocket disconnects, I reconnect using exponential backoff with a small random delay (jitter) to avoid overwhelming the server. I also use a heartbeat mechanism by periodically sending a ping and expecting a pong response. If no response is received within a certain time, I assume the connection is stale, close it, and let the reconnection logic establish a new connection. After reconnecting, I re-authenticate or re-subscribe to the required channels so the app continues working seamlessly.

CODE · TypeScript

```
function connectWithBackoff(url: string) {
  let attempt = 0;
  let pingTimer: ReturnType<typeof setInterval>;
  const open = () => {
    const ws = new WebSocket(url);
    ws.onopen = () => {
      attempt = 0;
      pingTimer = setInterval(() => {
        if (ws.readyState === WebSocket.OPEN) ws.send('{"type":"ping"}');
      }, 25000);
    };
    ws.onclose = () => {
      clearInterval(pingTimer);
      const base = Math.min(1000 * 2 ** attempt++, 30000); // cap 30s
      const delay = base / 2 + Math.random() * (base / 2);  // jitter
      setTimeout(open, delay);
    };
    return ws;
  };
  return open();
}
```

ARCHITECTURE / FLOW

```
Reconnect backoff with jitter
  close -> wait ~1s  -> try
  close -> wait ~2s  -> try
  close -> wait ~4s  -> try
  close -> wait ~8s  -> try  ... capped at 30s
  Heartbeat:
    C --ping--> S    (every 25s)
    C <--pong-- S    no pong in window -> force close -> backoff
```

Q65 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

What is a CRDT and what problem does it solve in a collaborative app?

THEORY

A CRDT (Conflict-free Replicated Data Type) is a data structure designed so that concurrent edits from multiple replicas can always be merged automatically without coordination or a central lock. Merges are commutative, associative, and idempotent, meaning order and duplication don't matter, so every replica converges to the same final state. This avoids the classic problem of last-write-wins clobbering someone's changes or needing server-side locks.

STRONG ANSWER

A CRDT (Conflict-free Replicated Data Type) is a data structure that allows multiple users to edit the same data at the same time, even while offline. When their changes are synchronized later, the CRDT automatically merges them without losing anyone's changes. It's commonly used in collaborative apps like shared documents, whiteboards, or note-taking apps. Instead of manually resolving conflicts, the CRDT ensures all users eventually see the same final state.

CODE · TypeScript

```
// Why last-write-wins fails vs why CRDTs converge
// LWW: two offline edits to same field
//   A sets title = 'Roadmap'   @ t1
//   B sets title = 'Q3 Plan'   @ t2  -> A's edit lost
// CRDT text: each char has a unique id + position
// concurrent inserts are ORDERED deterministically, not dropped
import * as Y from 'yjs';
const a = new Y.Doc();
const b = new Y.Doc();
a.getText('t').insert(0, 'Hello');
b.getText('t').insert(0, 'Hi');
// exchange updates both ways:
Y.applyUpdate(b, Y.encodeStateAsUpdate(a));
Y.applyUpdate(a, Y.encodeStateAsUpdate(b));
console.log(a.getText('t').toString() === b.getText('t').toString());
// -> true (both converge to the same merged string)
```

ARCHITECTURE / FLOW

```
CRDT convergence
  Replica A        Replica B
  edit X           edit Y
      \               /
       \   exchange  /
        v           v
      merge(X,Y) == merge(Y,X)   (commutative)
            |
        SAME final state on both  (no lock, no loss)
```

Q66 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

Can you explain the basics of Yjs: shared types, updates, and providers?

THEORY

Yjs centers on a Y.Doc, a container holding shared types like `Y.Text`, `Y.Array`, and `Y.Map` that behave like normal data structures but are CRDT-backed. Changes produce compact binary updates that can be applied to any other doc to sync it, and updates are encoded as deltas of the document state. Providers are pluggable transports (such as y-websocket or y-webrtc) that move those updates between peers and often handle persistence and awareness.

STRONG ANSWER

Yjs is a CRDT library that makes real-time collaboration easy. The main object is a Y.Doc, which represents a shared document. Inside it, you use shared types like Y.Text, Y.Map, or Y.Array to store collaborative data. Whenever someone edits the data, Yjs creates a small update. These updates are sent to other users, and Yjs automatically merges them so everyone stays in sync. A provider is responsible for transporting those updates. For example, a WebSocket provider sends updates between clients through a server, allowing everyone in the same room to see changes in real time.

CODE · TypeScript

```ts
import * as Y from 'yjs';
import { WebsocketProvider } from 'y-websocket';
const doc = new Y.Doc();
// shared types behave like normal structures, but are CRDTs
const text = doc.getText('content');
const meta = doc.getMap('meta');
// provider = transport that syncs updates to all peers in the room
const provider = new WebsocketProvider(
  'wss://api.example.com', 'doc-room-42', doc
);
// observe remote + local changes
text.observe(() => render(text.toString()));
// local edits -> binary update -> broadcast by provider automatically
text.insert(0, 'Sprint goals: ');
meta.set('updatedAt', Date.now());
provider.on('status', (e: { status: string }) => console.log(e.status));
```

ARCHITECTURE / FLOW

```
Yjs pieces
   Y.Doc
    |-- Y.Text  (content)   <- shared CRDT types
    |-- Y.Map   (meta)
    |-- Y.Array (items)
        |
     mutate -> binary UPDATE (delta)
        |
   Provider (y-websocket) -- relays update -> all peers in room
```

Q67 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How did you implement live presence and typing indicators?

THEORY

Presence is ephemeral state that doesn't belong in the persisted document, things like cursor position, name, color, and an is-typing flag. Yjs handles this through its awareness protocol, a separate channel that broadcasts each client's transient state and automatically clears it when they disconnect. Typing indicators are usually a boolean in awareness, debounced so they flip off shortly after the user stops typing.

STRONG ANSWER

For live presence and typing indicators, I keep that data separate from the actual document because it's temporary and doesn't need to be saved. I use Yjs Awareness to share each user's presence, such as their name, cursor position, or online status. For typing indicators, I send an isTyping status while the user is typing and clear it after a short delay using a debounce timer. If a user disconnects, Yjs automatically removes their presence information, so there are no stale typing indicators or ghost users.

CODE · TypeScript

```ts
import { WebsocketProvider } from 'y-websocket';
const awareness = provider.awareness; // ephemeral, not in the doc
awareness.setLocalStateField('user', { name: 'Rinshad', color: '#3b82f6' });
let typingTimer: ReturnType<typeof setTimeout>;
function onKeyPress() {
  awareness.setLocalStateField('typing', true);
  clearTimeout(typingTimer);
  typingTimer = setTimeout(
    () => awareness.setLocalStateField('typing', false),
    1000 // debounce: clear shortly after typing stops
  );
```

CODE · TypeScript (continued)

```
}
// render everyone's presence; disconnected peers auto-removed
awareness.on('change', () => {
  const peers = Array.from(awareness.getStates().values());
  renderPresence(peers); // names, cursors, typing bubbles
});
```

ARCHITECTURE / FLOW

```
Awareness (ephemeral) vs Doc (persisted)
  Y.Doc        ---> persisted, merged forever (text, data)
  Awareness    ---> transient: cursor, name, typing flag
                    auto-cleared on disconnect
  keypress -> typing=true --(debounce 1s)--> typing=false
```

Q68 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How do conflict-free offline edits work, and how do they merge when the device comes back online?

THEORY

Because Yjs is a CRDT, an offline replica can keep accepting edits with no server contact, storing them locally. Each edit carries enough identity (client id + sequence) that it can be merged later regardless of what others did meanwhile. On reconnect, the client exchanges a compact state vector so each side sends only the updates the other is missing, and the merge is deterministic so all replicas converge. No edits are dropped and no manual conflict resolution is needed.

STRONG ANSWER

With Yjs, users can continue editing even when they're offline. Their changes are stored locally, so no work is lost. When the device reconnects, Yjs exchanges only the missing updates with the server and other clients. Because Yjs is based on CRDTs, it automatically merges offline and online changes without conflicts, ensuring every user eventually sees the same document.

CODE · TypeScript

```
import * as Y from 'yjs';
// while offline, edits accumulate locally and are persisted
function queueOfflineEdit(doc: Y.Doc) {
  doc.getText('content').insert(0, 'offline note ');
  localStorage.setItem('doc', JSON.stringify(
    Array.from(Y.encodeStateAsUpdate(doc))
  ));
}
// on reconnect: send only what the other side lacks (state vector diff)
function sync(local: Y.Doc, remoteStateVector: Uint8Array) {
  // give peer exactly the updates they're missing
  const diff = Y.encodeStateAsUpdate(local, remoteStateVector);
  ws.send(diff);
}
function applyRemote(local: Y.Doc, update: Uint8Array) {
  Y.applyUpdate(local, update); // deterministic merge, nothing lost
}
```

ARCHITECTURE / FLOW

```
Offline edit + merge on reconnect
  Device (offline)        Server / peers
   edit A                  edit B, edit C
   edit A2     [no network]
      |  reconnect: exchange state vectors
      |--- missing: A, A2 -------->
      |<-- missing: B, C ----------
      v
   merge(A,A2,B,C) deterministic -> identical state everywhere
```

Q69 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

Walk me through WebRTC basics, peer connection, signaling, and STUN/TURN, for sending voice notes.

THEORY

WebRTC sets up a direct peer-to-peer media/data channel between two clients. The peers first exchange SDP offer/answer describing codecs and media, plus ICE candidates (possible network paths), through a separate signaling channel you provide, often a WebSocket. STUN servers help a peer discover its public address behind NAT, and when direct connection fails, a TURN server relays the media as a fallback. The signaling server only brokers the setup; the actual audio flows peer-to-peer once connected.

STRONG ANSWER

WebRTC is a technology that lets two devices communicate directly for real-time audio, video, or data. The connection starts with signaling, where the two devices exchange information like offers, answers, and ICE candidates through a server, usually using WebSockets. Once both devices have that information, they create an RTCPeerConnection. To establish the best network path, WebRTC first tries a STUN server to discover each device's public IP address. If a direct connection isn't possible because of NAT or firewalls, it falls back to a TURN server, which relays the media. After the peer connection is established, the voice data flows directly between the devices whenever possible, giving low latency and reducing server load.

CODE · TypeScript

```typescript
const pc = new RTCPeerConnection({
  iceServers: [
    { urls: 'stun:stun.l.google.com:19302' },
    { urls: 'turn:turn.example.com', username: 'u', credential: 'p' },
  ],
});
// attach mic for the voice note
const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
stream.getTracks().forEach((t) => pc.addTrack(t, stream));
// signaling over the existing WebSocket
pc.onicecandidate = (e) => e.candidate && signal({ ice: e.candidate });
// caller side
const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
signal({ sdp: offer }); // send via WS; peer replies with an answer
```

ARCHITECTURE / FLOW

```
WebRTC setup
  Peer A               Signaling (WS)           Peer B
    |-- offer (SDP) ------->  |  -- offer -------->  |
    |<------- answer (SDP) -- |  <-- answer -------- |
    |-- ICE candidates ---->  | <--- ICE ---------->  |
  Path discovery:
```

ARCHITECTURE / FLOW (continued)

```
    STUN -> learn public IP (direct P2P if possible)
    TURN -> relay audio when direct path is blocked
  Once connected: AUDIO flows A <==P2P==> B (not via server)
```

Q70 REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How do you scale a real-time backend across many servers, and why do you need Redis or a message broker?

THEORY

A single WebSocket server only knows about the clients connected to it, so once you run multiple instances behind a load balancer, a broadcast on one server won't reach users on another. The fix is a pub/sub layer (commonly Redis Pub/Sub or a broker like NATS/Kafka) where each server publishes room events and all servers subscribe, fanning messages out to their local sockets. Rooms scope messages so only relevant clients get them, and sticky sessions or a shared store keep connection state coherent. This decouples message routing from any single node.

STRONG ANSWER

When you have multiple backend servers, each server only knows about the clients connected to it. If one user sends a real-time message, the other users might be connected to different servers. To solve this, all servers communicate through a shared messaging system like Redis Pub/Sub. When one server receives an event, it publishes it to Redis, and the other servers receive it and forward it to their connected clients. For very large systems that need message persistence and guaranteed delivery, I'd use a message broker like Kafka instead of Redis.

CODE · TypeScript

```typescript
import { createClient } from 'redis';
const pub = createClient();
const sub = pub.duplicate();
await pub.connect();
await sub.connect();
// each server subscribes to room channels
await sub.pSubscribe('room:*', (message, channel) => {
  const room = channel.split(':')[1];
  // deliver to sockets connected to THIS server in that room
  localSockets.get(room)?.forEach((ws) => ws.send(message));
});
// when a client sends an edit, publish so ALL servers fan it out
function onClientMessage(room: string, payload: string) {
  pub.publish(`room:${room}`, payload);
}
```

ARCHITECTURE / FLOW

```
Scaling fan-out via pub/sub
   Client A --- WS ---> Server 1        Server 2 <--- WS --- Client B
                          |                 |
                          v                 v
                    publish room:42   subscribe room:42
                         \                /
                          >  Redis Pub/Sub  <
   (Server 1 publishes; Server 2 receives and pushes to Client B)
   Rooms scope delivery; broker decouples routing from any node
8
AI / LLM Features on Mobile
Streaming chat, on-device Whisper, tool calling, RAG, embeddings & VAD.
Q71–Q80
8
AI / LLM Features on Mobile
```

ARCHITECTURE / FLOW (continued)

```
Streaming chat, on-device Whisper, tool calling, RAG, embeddings & VAD.
Q71–Q80
8
AI / LLM Features on Mobile
Streaming chat, on-device Whisper, tool calling, RAG, embeddings & VAD.
Q71–Q80

8
AI / LLM Features on Mobile
Streaming chat, on-device Whisper, tool calling, RAG, embeddings & VAD.
Q71–Q80
```

8 AI / LLM Features on Mobile

Streaming chat, on-device Whisper, tool calling, RAG, embeddings & VAD.

Q71–Q80

Q71 AI / LLM FEATURES ON MOBILE

How does LLM streaming work, and why would you stream a response instead of waiting for the full reply?

THEORY

LLMs generate text one token at a time (a token is roughly a word-piece), so the server can emit each token the moment it is produced rather than buffering the whole answer. Most APIs deliver this over Server-Sent Events (SSE), a long-lived HTTP response where the server pushes `data:` lines until a final `[DONE]` marker. Streaming cuts perceived latency dramatically because the user sees the first words in a few hundred milliseconds instead of waiting several seconds for a full paragraph.

STRONG ANSWER

LLM streaming means the AI sends its response one small piece (or token) at a time instead of waiting until the entire answer is generated. The client receives these tokens continuously and updates the UI in real time. I prefer streaming because it greatly improves the user experience. Users see the response almost immediately instead of waiting several seconds for the complete answer. It's especially useful for chat apps and AI assistants, where responsiveness is important.

CODE · TypeScript

```typescript
// Anatomy of an SSE token stream from an LLM endpoint
// Each event line looks like:
//
//   data: {"choices":[{"delta":{"content":"Hel"}}]}
//   data: {"choices":[{"delta":{"content":"lo"}}]}
//   data: [DONE]
function parseSSEChunk(raw: string): string[] {
  const tokens: string[] = [];
  for (const line of raw.split("\n")) {
    if (!line.startsWith("data:")) continue;
    const payload = line.slice(5).trim();
    if (payload === "[DONE]") break;
    try {
      const json = JSON.parse(payload);
      const delta = json.choices?.[0]?.delta?.content;
      if (delta) tokens.push(delta);
    } catch {
      // partial JSON across chunk boundary; buffer & retry upstream
    }
  }
  return tokens;
}
```

ARCHITECTURE / FLOW

```
Non-streaming:  request --------[ wait 4s ]--------> full reply
Streaming:      request -> tok -> tok -> tok -> ... -> [DONE]
                          ^first token ~300ms (UI updates live)
```

Q72 AI / LLM FEATURES ON MOBILE

How do you implement streaming chat in React Native, consuming the stream and rendering tokens as they arrive?

THEORY

React Native's default `fetch` does not expose a streaming body reader on all platforms, so the common approach is a library like react-native-sse or `fetch` with XHR-based streaming to read `data:` events as they come. As each token arrives you append it to state and let React re-render the growing message. To avoid jank you batch updates (or use a ref + throttled flush) so you aren't calling `setState` on every single token.

STRONG ANSWER

In React Native, I use Server-Sent Events (SSE) to receive the AI response as it's generated. I open the stream, listen for incoming messages, and append each chunk of text to the current assistant message so the response appears gradually, just like ChatGPT. To keep the UI smooth, I avoid updating React state for every token. Instead, I batch incoming tokens and update the UI at short intervals. I also handle errors, close the stream when it's finished, and allow the user to cancel the response if needed.

CODE · TypeScript

```
import EventSource from "react-native-sse";
function streamChat(prompt: string, onToken: (t: string) => void) {
  const es = new EventSource("https://api.groq.com/openai/v1/chat/completions", {
    method: "POST",
    headers: { Authorization: `Bearer ${KEY}`, "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "llama-3.1-8b-instant",
      stream: true,
      messages: [{ role: "user", content: prompt }],
    }),
  });
  es.addEventListener("message", (e) => {
    if (e.data === "[DONE]") return es.close();
    const delta = JSON.parse(e.data!).choices?.[0]?.delta?.content;
    if (delta) onToken(delta); // append to buffered ref, flush on rAF
  });
  es.addEventListener("error", () => es.close());
  return () => es.close(); // call to cancel an in-flight stream
}
```

ARCHITECTURE / FLOW

```
SSE event  ->  onToken()  ->  buffer ref  --(throttled flush)-->
   setState  ->  FlatList re-render  ->  bubble grows live
```

Q73 AI / LLM FEATURES ON MOBILE

Why run speech-to-text on-device with Whisper (whisper.cpp) instead of calling a cloud transcription API?

THEORY

Whisper is OpenAI's open speech recognition model, and whisper.cpp is a C/C++ port that runs quantized model weights efficiently on a phone's CPU/GPU. Running it on-device means audio never leaves the phone, which is a privacy win and removes network round-trips for lower latency. It also keeps cost at zero (no per-minute API billing) and works offline, at the price of bundling a model file and using more battery/CPU.

STRONG ANSWER

I prefer on-device speech-to-text with Whisper because it's faster, more private, and works even without an internet connection. Since the audio is processed locally, the user's voice never leaves the device, which improves privacy and reduces network latency. The trade-off is that the model increases the app size and uses more CPU, so I choose a smaller, optimized model that provides a good balance between speed and accuracy.

CODE · TypeScript

```
import { initWhisper } from "whisper.rn";
let ctx: Awaited<ReturnType<typeof initWhisper>>;
export async function loadModel() {
  // quantized ggml model bundled with the app
  ctx = await initWhisper({ filePath: require("../assets/ggml-base.q5.bin") });
}
export async function transcribe(wavPath: string): Promise<string> {
  const { promise } = ctx.transcribe(wavPath, {
    language: "en",
    maxThreads: 4,    // tune for the device's core count
    translate: false,
  });
  const { result } = await promise; // runs fully offline, no network
  return result.trim();
}
```

ARCHITECTURE / FLOW

```
Cloud STT:  mic -> upload audio -> server -> text   (privacy + RTT cost)
On-device:  mic -> whisper.cpp (phone CPU) -> text  (private, offline, $0)
```

Q74 AI / LLM FEATURES ON MOBILE

How do you handle text-to-speech playback so the assistant can speak its replies?

THEORY

Text-to-speech (TTS) converts the model's text answer into audio. On mobile you can use the OS-native TTS engine (via expo-speech or react-native-tts) for zero cost and offline support, or a cloud TTS for more natural voices at the price of latency and billing. A key UX trick with streaming LLMs is to speak sentence-by-sentence as tokens arrive rather than waiting for the full reply, so the assistant starts talking almost immediately.

STRONG ANSWER

For text-to-speech, I use the device's native TTS engine to convert the AI's response into speech. As the AI generates its response, I queue complete sentences for playback instead of waiting for the entire response. This allows the assistant to start speaking almost immediately, making the conversation feel more natural. I also manage playback properly by stopping the current speech if the user interrupts or starts speaking again, ensuring a smooth voice interaction.

CODE · TypeScript

```
import * as Speech from "expo-speech";
let buffer = "";
// Feed streaming LLM tokens; speak each completed sentence immediately.
export function feedTokenToTTS(token: string) {
  buffer += token;
  const match = buffer.match(/^(.*?[.!?])\s+(.*)$/s);
  if (match) {
    const [, sentence, rest] = match;
    Speech.speak(sentence, { language: "en", rate: 1.0 });
```

CODE · TypeScript (continued)

```
    buffer = rest; // keep remainder for the next sentence
  }
}
export function flushTTS() {
  if (buffer.trim()) Speech.speak(buffer.trim());
  buffer = "";
}
export function stopTTS() {
  Speech.stop(); // call when user starts speaking (barge-in)
}
```

Q75 AI / LLM FEATURES ON MOBILE

What is tool calling (function calling), and how do you wire it up with an LLM?

THEORY

Tool calling lets the model request a function instead of answering directly: you describe your tools as a JSON schema (name, description, parameters), and when the model decides one is needed it returns a structured `tool_call` with arguments. Your app executes the real function, feeds the result back into the conversation, and the model uses it to produce the final reply. This is how an LLM safely reaches out to the live world (weather, search, your database) without hallucinating the data.

STRONG ANSWER

Tool calling, also called function calling, allows an LLM to use external functions or APIs when it needs information it doesn't already know. Instead of guessing an answer, the model requests a tool with the required parameters. My application validates those parameters, executes the actual function or API call, sends the result back to the model, and then the model generates the final response using that real data.

CODE · TypeScript

```
const tools = [{
  type: "function",
  function: {
    name: "get_reminders",
    description: "Fetch the user's reminders for a given day",
    parameters: {
      type: "object",
      properties: { date: { type: "string", description: "ISO date" } },
      required: ["date"],
    },
  },
}];
async function handleToolCalls(call: { name: string; arguments: string }) {
  const args = JSON.parse(call.arguments); // validate before trusting!
  if (call.name === "get_reminders") {
    const data = await db.getReminders(args.date);
    return { role: "tool", name: call.name, content: JSON.stringify(data) };
  }
  throw new Error(`Unknown tool: ${call.name}`);
}
```

ARCHITECTURE / FLOW

```
user -> LLM --(needs data)--> tool_call{name,args}
          ^                          |
          |                     app runs fn
      final answer <-- LLM <-- tool result (fed back)
```

Q76 AI / LLM FEATURES ON MOBILE

What is RAG (Retrieval-Augmented Generation), and why do you need it?

THEORY

RAG means retrieving relevant chunks of your own knowledge base and injecting them into the prompt so the model answers from real, current data instead of only its training. You need it because LLMs have a fixed knowledge cutoff, don't know your private content, and can hallucinate when asked about things outside their training. By grounding the answer in retrieved context you get accurate, source-backed responses without retraining the model.

STRONG ANSWER

RAG (Retrieval-Augmented Generation) is a technique where an LLM retrieves relevant information from an external knowledge source before generating an answer. Instead of relying only on what the model learned during training, it uses real, up-to-date data to answer questions. It's useful because it reduces hallucinations, allows the AI to answer questions about private or company-specific documents, and keeps responses accurate without retraining the model.

CODE · TypeScript

```typescript
// RAG query flow: embed -> search -> assemble -> generate
async function answerWithRAG(question: string) {
  const qVec = await embed(question);                // Gemini embedding
  const chunks = await searchSimilar(qVec, 5);       // pgvector cosine
  const context = chunks.map((c, i) => `[${i + 1}] ${c.text}`).join("\n\n");
  const messages = [
    { role: "system", content:
      "Answer ONLY from the context below. If unknown, say so.\n\n" + context },
    { role: "user", content: question },
  ];
  return streamChat(messages); // grounded, source-backed answer
}
```

ARCHITECTURE / FLOW

```
ingest:  docs -> chunk -> embed -> store vectors (pgvector)
query:   question -> embed -> vector search -> top-k chunks
             -> inject into prompt -> LLM -> grounded answer
```

Q77 AI / LLM FEATURES ON MOBILE

How do embeddings and vector similarity search work, and what do cosine distance, pgvector, and an HNSW index give you?

THEORY

An embedding maps text to a high-dimensional vector so that semantically similar text lands close together in that space. Cosine similarity measures the angle between two vectors, so it ranks by meaning regardless of length, and finding the closest stored vectors is a nearest-neighbor search. pgvector adds a `vector` column type to PostgreSQL, and an HNSW index (a graph-based approximate-nearest-neighbor structure) makes that search fast at scale instead of scanning every row.

STRONG ANSWER

Embeddings are numerical representations of text that capture its meaning. Similar texts have similar embeddings. When a user asks a question, I convert it into an embedding and compare it with the stored document embeddings to find the most relevant content. For similarity search, I typically use cosine similarity because it measures how semantically close two embeddings are. I store the embeddings in a vector database such as PostgreSQL with pgvector. As the dataset grows, I use an HNSW index to make nearest-neighbor searches much faster while maintaining high accuracy.

CODE · SQL

```sql
-- pgvector table with an HNSW index for fast cosine search
CREATE TABLE chunks (
  id      bigserial PRIMARY KEY,
  text    text,
  embedding vector(768)        -- Gemini embedding dimension
);
CREATE INDEX ON chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
-- top-5 most similar chunks to the query vector ($1)
SET hnsw.ef_search = 40;       -- higher = better recall, slower
SELECT id, text, 1 - (embedding <=> $1) AS similarity
FROM chunks
ORDER BY embedding <=> $1       -- <=> is cosine distance
LIMIT 5;
```

ARCHITECTURE / FLOW

```
vector space (meaning)
   query *                           cat o
         \  cosine angle small        /
          \---- nearest neighbors ----> kitten o
           \                           puppy o
   HNSW graph hops between nodes -> finds top-k fast
```

Q78 AI / LLM FEATURES ON MOBILE

What are the basics of prompt engineering you rely on — system prompt, context injection, and token limits?

THEORY

The system prompt sets the model's role, rules, and tone for the whole conversation and is the strongest lever on behavior. Context injection means inserting retrieved data or conversation history into the prompt so the model has the facts it needs at generation time. Every model has a context window measured in tokens, so you must budget how much system text, history, and retrieved context you include, or older content gets truncated and the model loses track.

STRONG ANSWER

When working with LLMs, I focus on three things: a clear system prompt, relevant context, and token limits. The system prompt defines the assistant's role and behavior. Before sending a user question, I inject only the relevant context, such as retrieved documents or recent conversation, so the model has the information it needs. I also keep the prompt within the model's token limit by including only the most relevant context or summarizing older messages. This improves accuracy, reduces cost, and keeps responses fast.

CODE · TypeScript

```
const MAX_CONTEXT_TOKENS = 6000;
const est = (s: string) => Math.ceil(s.length / 4); // rough token estimate
function buildPrompt(system: string, chunks: string[], history: Msg[], q: string) {
  let budget = MAX_CONTEXT_TOKENS - est(system) - est(q);
  const ctx: string[] = [];
  for (const c of chunks) {            // inject highest-ranked chunks first
    if (budget - est(c) < 0) break;
    ctx.push(c); budget -= est(c);
  }
  const recent: Msg[] = [];
  for (let i = history.length - 1; i >= 0; i--) {  // newest history that fits
    if (budget - est(history[i].content) < 0) break;
    recent.unshift(history[i]); budget -= est(history[i].content);
  }
  return [{ role: "system", content: `${system}\n\nContext:\n${ctx.join("\n")}` },
          ...recent, { role: "user", content: q }];
}
```

ARCHITECTURE / FLOW

```
[ context window budget (tokens) ]
| system prompt | retrieved context | history | user q |
   ^fixed rules    ^top-k chunks      ^trim old  ^always kept
   if it overflows -> oldest history dropped first
```

Q79 AI / LLM FEATURES ON MOBILE

How do you manage latency and cost on free-tier APIs like Gemini and Groq, including caching?

THEORY

Free-tier APIs impose rate limits and quotas (requests/tokens per minute), so you must avoid redundant calls and degrade gracefully when throttled. Caching identical or near-identical requests, streaming to lower perceived latency, and routing simple work to the fastest/cheapest model are the main levers. Trimming prompt size and embedding-caching also directly reduce both token cost and round-trip time.

STRONG ANSWER

When using free-tier LLM APIs, I focus on reducing unnecessary API calls and keeping responses fast. I cache embeddings and repeated responses so I don't make the same request twice. I also keep prompts concise to reduce token usage and cost. For a better user experience, I use streaming so users see responses immediately. I also handle rate limits by retrying with exponential backoff, and if one provider is unavailable, I fall back to another provider whenever possible.

CODE · TypeScript

```
const cache = new Map<string, string>();         // prompt -> response
async function cachedComplete(prompt: string): Promise<string> {
  const key = hash(prompt);
  if (cache.has(key)) return cache.get(key)!;     // skip the API entirely
  for (let attempt = 0; attempt < 4; attempt++) {
    const res = await fetch(GROQ_URL, { method: "POST", body: bodyFor(prompt) });
    if (res.status === 429) {                       // rate limited
      await sleep(2 ** attempt * 500);              // exponential backoff
      continue;
    }
    const text = (await res.json()).choices[0].message.content;
    cache.set(key, text);
    return text;
```

CODE · TypeScript (continued)

```
  }
  throw new Error("rate limited; fall back to cached/older model");
}
```

ARCHITECTURE / FLOW

```
request -> [cache hit?] --yes--> instant, $0
               |no
               v
          API call -> 429? -> backoff & retry -> cache result
```

Q80 AI / LLM FEATURES ON MOBILE

What is Voice Activity Detection (VAD), and how does the end-to-end voice pipeline fit together (mic to VAD to Whisper to LLM to TTS)?

THEORY

Voice Activity Detection distinguishes speech from silence/noise in the mic stream, so you know when the user starts and stops talking without a push-to-talk button. It triggers transcription only on real speech, which saves CPU and avoids feeding silence to Whisper. In the full loop, VAD gates the audio, Whisper turns speech into text, the LLM generates a streamed answer, and TTS speaks it back, ideally with barge-in so the user can interrupt.

STRONG ANSWER

Voice Activity Detection (VAD) detects when a user starts and stops speaking. Instead of recording continuously or requiring a button press, it automatically captures only the spoken audio. Once the user finishes speaking, the recorded audio is sent to Whisper for speech-to-text. The transcribed text is then sent to the LLM, which generates a response. Finally, the response is converted into speech using a TTS engine. If the user starts speaking again while the assistant is talking, I stop the current speech so the conversation feels natural.

CODE · TypeScript

```
// Orchestrating the voice loop, gated by VAD events
vad.on("speechStart", () => {
  stopTTS();          // barge-in: user interrupts the assistant
  cancelStream?.();   // abort any in-flight LLM stream
});
vad.on("speechEnd", async (segment: AudioSegment) => {
  const text = await transcribe(segment.wavPath);   // on-device Whisper
  if (!text) return;
  cancelStream = streamChat(buildPrompt(text), (token) => {
    appendToBubble(token);   // render live
    feedTokenToTTS(token);   // speak sentence-by-sentence
  });
});
```

ARCHITECTURE / FLOW

```
mic stream
     |
   [ VAD ] --silence--> ignore
     | speech end
     v
 [ Whisper STT ] -> text -> [ LLM (stream) ] -> tokens
                                |                  |
                           appendToBubble      [ TTS ] -> speaker
   user speaks again -> VAD speechStart -> barge-in: stop TTS+stream
9
```

ARCHITECTURE / FLOW (continued)

```
Backend & Data
REST, Express, JWT rotation, SQL vs NoSQL, indexing, Redis & idempotency.
Q81–Q90
9
Backend & Data
REST, Express, JWT rotation, SQL vs NoSQL, indexing, Redis & idempotency.
Q81–Q90
9
Backend & Data
REST, Express, JWT rotation, SQL vs NoSQL, indexing, Redis & idempotency.
Q81–Q90

9
Backend & Data
REST, Express, JWT rotation, SQL vs NoSQL, indexing, Redis & idempotency.
Q81–Q90
```

9 Backend & Data

REST, Express, JWT rotation, SQL vs NoSQL, indexing, Redis & idempotency.

Q81–Q90

Q81 BACKEND & DATA

How do you design a clean REST API in terms of resources, HTTP verbs, and status codes?

THEORY

REST models the system as resources addressed by nouns in the URL, not verbs. The HTTP method conveys the action: GET reads, POST creates, PUT/PATCH update, DELETE removes. Status codes communicate outcome so clients can react programmatically without parsing the body: 2xx success, 4xx client error, 5xx server error.

STRONG ANSWER

I design REST APIs around resources, not actions. For example, I use /users or /orders instead of URLs like /getUsers. The HTTP method defines the action: GET for reading, POST for creating, PUT or PATCH for updating, and DELETE for removing data. I also use appropriate HTTP status codes, such as 200 for success, 201 for creation, 400 for invalid requests, 401 for unauthorized access, 403 for forbidden access, 404 when a resource isn't found, and 500 for server errors. Keeping response formats and status codes consistent makes the frontend easier to build and maintain.

CODE · TypeScript

```typescript
// Resource-oriented Express routes
import { Router } from 'express';
const router = Router();
router.get('/users/:id/orders', async (req, res) => {
  const orders = await orderService.listByUser(req.params.id);
  return res.status(200).json({ data: orders });
});
router.post('/users/:id/orders', async (req, res) => {
  const order = await orderService.create(req.params.id, req.body);
  // 201 + Location header for the new resource
  res.location(`/orders/${order.id}`);
  return res.status(201).json({ data: order });
});
router.delete('/orders/:orderId', async (req, res) => {
  const removed = await orderService.remove(req.params.orderId);
  if (!removed) return res.status(404).json({ error: 'Not found' });
  return res.status(204).send(); // no content
});
export default router;
```

ARCHITECTURE / FLOW

```
Verb + Resource -> Status
  GET    /users/:id/orders     -> 200 list
  POST   /users/:id/orders     -> 201 created (+Location)
  PATCH  /orders/:id           -> 200 updated
  DELETE /orders/:id           -> 204 / 404 if missing
  (bad body 400 | no token 401 | forbidden 403)
```

Q82 BACKEND & DATA

What is Express middleware and how does the request flow through it?

THEORY

Express processes each request through an ordered chain of middleware functions, each receiving (req, res, next). A middleware can run logic, mutate req/res, end the response, or call next() to pass control to the next handler. Order matters: middleware registered earlier runs first, which is why auth and body parsing sit before route handlers and error handlers sit last.

STRONG ANSWER

Express middleware is a function that runs before a request reaches the route handler. It can inspect or modify the request, send a response, or pass control to the next middleware using next(). I use middleware for common tasks like parsing JSON, authentication, logging, validation, and error handling. This keeps my route handlers focused on business logic and makes the code more reusable and easier to maintain.

CODE · TypeScript

```typescript
import express, { Request, Response, NextFunction } from 'express';
const app = express();
app.use(express.json()); // body parser middleware
// custom logging middleware
app.use((req: Request, _res: Response, next: NextFunction) => {
  console.log(`${req.method} ${req.path}`);
  next(); // pass control onward
});
function requireAuth(req: Request, res: Response, next: NextFunction) {
  if (!req.headers.authorization) return res.status(401).json({ error: 'No token' });
  next();
}
app.get('/profile', requireAuth, (req, res) => {
  res.json({ data: { id: 1 } });
});
// error-handling middleware (4 args) registered LAST
app.use((err: Error, _req: Request, res: Response, _next: NextFunction) => {
  res.status(500).json({ error: err.message });
});
```

ARCHITECTURE / FLOW

```
Request
   |
   v
[express.json] -> [logger] -> [requireAuth] -> [controller]
   |                                              |
   |  (any throw / next(err))                     v
   +----------------------> [error handler] <-- Response
```

Q83 BACKEND & DATA

What is the difference between access tokens and refresh tokens in JWT authentication?

THEORY

An access token is short-lived (minutes) and is sent on every API call to prove identity; because it expires fast, a leaked one has a small blast radius. A refresh token is long-lived and used only to mint new access tokens, so it is stored more carefully and never sent on normal requests. This split lets you keep sessions long without keeping a powerful credential in flight constantly.

STRONG ANSWER

In JWT authentication, an access token is a short-lived token that's sent with every API request to prove the user is authenticated. A refresh token is long-lived and is used only to request a new access token when the current one expires, so the user doesn't have to log in again. For security, I store the refresh token in secure storage, such as the Keychain on iOS or the Keystore on Android, and keep the access token in memory since it expires quickly.

CODE · TypeScript

```
import jwt from 'jsonwebtoken';
const ACCESS_TTL = '15m';
const REFRESH_TTL = '30d';
export function issueTokens(userId: string) {
  const accessToken = jwt.sign({ sub: userId }, process.env.ACCESS_SECRET!, {
    expiresIn: ACCESS_TTL,
  });
  const refreshToken = jwt.sign({ sub: userId, typ: 'refresh' }, process.env.REFRESH_SECRET!, {
    expiresIn: REFRESH_TTL,
  });
  return { accessToken, refreshToken };
}
// middleware verifying the access token on each request
export function verifyAccess(req: any, res: any, next: any) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Missing token' });
  try {
    req.user = jwt.verify(token, process.env.ACCESS_SECRET!);
    next();
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

ARCHITECTURE / FLOW

```
Login -> { access(15m), refresh(30d) }
Normal call:  Authorization: Bearer <access>  -> verify sig
Access expired (401):
  client -> POST /refresh  (refresh token)
         <- new { access, refresh }
  retry original request with new access
```

Q84 BACKEND & DATA

How does refresh-token rotation with token families help detect token theft?

THEORY

With rotation, every use of a refresh token invalidates it and issues a brand-new one, so a refresh token is single-use. All tokens descending from one login share a family ID, letting the server reason about the whole lineage. If an already-used (revoked) refresh token is replayed, that signals theft, and the server revokes the entire family, logging out both attacker and victim.

STRONG ANSWER

Refresh-token rotation means that every time a client uses a refresh token, the server invalidates it and issues a brand-new refresh token. This ensures that a refresh token can only be used once. Each refresh token belongs to a token family that represents a user's session. If an old refresh token is used again after it has already been rotated, the server knows that the token has likely been stolen or replayed. In that case, it revokes the entire token family, forcing the user to log in again and preventing an attacker from continuing to refresh access tokens.

CODE · TypeScript

```
import crypto from 'crypto';
const hash = (t: string) => crypto.createHash('sha256').update(t).digest('hex');
export async function rotateRefresh(presented: string) {
  const payload = jwt.verify(presented, process.env.REFRESH_SECRET!) as any;
  const row = await db.refreshTokens.findByHash(hash(presented));
  // replay of an already-used token => theft
  if (!row || row.used) {
    if (row) await db.refreshTokens.revokeFamily(row.familyId);
    throw new Error('Refresh reuse detected - family revoked');
  }
  await db.refreshTokens.markUsed(row.id); // burn old token
  const next = issueTokens(payload.sub);
  await db.refreshTokens.insert({
    familyId: row.familyId,            // same lineage
    tokenHash: hash(next.refreshToken),
    used: false,
  });
  return next;
}
```

ARCHITECTURE / FLOW

```
Login => family F1, token R0
  use R0 -> R0 marked used, issue R1
  use R1 -> R1 marked used, issue R2  ...
Thief replays R0 (already used):
  server sees used=true  -> REVOKE family F1
  both attacker & victim must log in again
```

Q85 BACKEND & DATA

How should passwords be hashed and application secrets be stored securely?

THEORY

Passwords must never be stored reversibly; you use a slow, salted, adaptive hash like bcrypt, scrypt, or Argon2 so brute-forcing is expensive and identical passwords get different hashes. The per-user salt is built into these algorithms, defeating rainbow tables. Application secrets (signing keys, DB passwords) belong in environment variables or a secrets manager, never in source control.

STRONG ANSWER

Passwords should never be stored in plain text. I hash them using a strong password hashing algorithm like bcrypt, which automatically adds a unique salt and is intentionally slow to make brute-force attacks difficult. I store only the hash in the database and verify passwords using bcrypt.compare() during login. For application secrets, such as JWT signing keys or database credentials, I never hardcode them in the source code. Instead, I store them securely in environment variables or a secrets manager, use different secrets for different environments, and rotate them periodically. I also keep access-token and refresh-token signing keys separate so compromising one doesn't automatically compromise the other.

CODE · TypeScript

```
import bcrypt from 'bcrypt';
const COST = 12; // adaptive work factor
export async function registerUser(email: string, plain: string) {
  const passwordHash = await bcrypt.hash(plain, COST); // salt is embedded
  return db.users.insert({ email, passwordHash });
}
export async function verifyLogin(email: string, plain: string) {
  const user = await db.users.findByEmail(email);
  // compare even when user missing to avoid timing/enumeration leaks
```

CODE · TypeScript (continued)

```
  const ok = await bcrypt.compare(plain, user?.passwordHash ?? '$2b$12$invalidplaceholderhashvaluehere');
  if (!user || !ok) throw new Error('Invalid credentials');
  return user;
}
// secrets come from env, never hardcoded
const ACCESS_SECRET = process.env.ACCESS_SECRET;
if (!ACCESS_SECRET) throw new Error('Missing ACCESS_SECRET');
```

Q86 BACKEND & DATA

When would you choose PostgreSQL over MongoDB, and vice versa?

THEORY

PostgreSQL is a relational store with strong schemas, joins, and ACID transactions, ideal when data is structured and relationships and consistency matter. MongoDB is a document store that shines when the shape varies or you want to read a whole nested object in one hit without joins. The choice is about access patterns and consistency needs, not which is newer or faster in the abstract.

STRONG ANSWER

I choose PostgreSQL when the data is highly relational and consistency is important. It's ideal for applications like e-commerce, banking, or user management because it provides ACID transactions, foreign keys, joins, and powerful SQL queries. I choose MongoDB when the data is document-oriented or the schema changes frequently. It's a good fit for content management systems, logs, event data, or applications where each record is mostly self-contained. The choice depends on the data model, query patterns, and consistency requirements rather than which database is generally better.

CODE · SQL

```
-- PostgreSQL: relational integrity + transaction
BEGIN;
INSERT INTO orders (id, user_id, total_cents)
VALUES ('o_1', 'u_9', 4200);
UPDATE accounts SET balance_cents = balance_cents - 4200
WHERE user_id = 'u_9' AND balance_cents >= 4200;
-- foreign key guarantees the user exists; rollback on any failure
COMMIT;
-- pgvector similarity search alongside relational data
SELECT id, title
FROM documents
ORDER BY embedding <-> '[0.12, 0.04, ...]'::vector
LIMIT 5;
```

ARCHITECTURE / FLOW

```
PostgreSQL                  MongoDB
  +-----------+               +------------------------+
  | users     |<--FK--+       | { _id, user, items:[   |
  | orders    |       |       |   {sku, qty} ], meta{} |
  | accounts  |       |       | }   (one nested doc)   |
  +-----------+               +------------------------+
  structured + ACID + joins   flexible schema, self-contained
  -> source of truth          -> high-churn / variable shape
```

Q87 BACKEND & DATA

How did you use indexing and query optimization to cut API latency by 35%?

THEORY

An index is a sorted lookup structure that lets the database find rows without scanning the whole table, turning O(n) scans into O(log n) seeks. You target indexes at columns used in WHERE, JOIN, and ORDER BY, and use EXPLAIN ANALYZE to confirm the planner actually uses them. Beware over-indexing, which slows writes, and the N+1 query pattern, which a single batched query usually fixes.

STRONG ANSWER

I always start by measuring before optimizing. I identify slow queries using query logs and EXPLAIN ANALYZE to understand whether the database is using indexes or performing full table scans. For the slowest queries, I added indexes that matched the application's filtering and sorting patterns, eliminated N+1 queries by replacing multiple database calls with joins or batched queries, and cached read-heavy data in Redis. After each change, I measured the performance again to verify the improvement. Together, these optimizations reduced API latency by about 35%.

CODE · SQL

```sql
-- before: seq scan on a large table, filtered + sorted
EXPLAIN ANALYZE
SELECT id, total_cents, created_at
FROM orders
WHERE user_id = 'u_9' AND status = 'active'
ORDER BY created_at DESC
LIMIT 20;
-- composite index matching WHERE columns then ORDER BY column
CREATE INDEX idx_orders_user_status_created
ON orders (user_id, status, created_at DESC);
-- partial index: most reads only touch active rows
CREATE INDEX idx_orders_active
ON orders (user_id, created_at DESC)
WHERE status = 'active';
-- re-run EXPLAIN ANALYZE: expect Index Scan, not Seq Scan
```

ARCHITECTURE / FLOW

```
Without index: Seq Scan -> read every row -> O(n)
  [r1][r2][r3].........................[rN]   slow
With composite index (user_id,status,created_at):
  B-tree seek -> jump to matching range -> read 20
  O(log n) + small range scan            fast
  Result: hot endpoints ms-level, ~35% latency drop
```

Q88 BACKEND & DATA

What are the main Redis use cases, and how do you build a multi-instance-safe rate limiter with it?

THEORY

Redis is an in-memory store used for caching, session storage, queues, and rate limiting because reads and writes are sub-millisecond. For rate limiting across many app instances, the counter must live in one shared place (Redis), not per-process memory, or each instance enforces its own limit. To stay correct under concurrency you run the increment and the limit check atomically, typically via a Lua script or INCR with EXPIRE.

STRONG ANSWER

Redis is an in-memory data store that I use for caching, session storage, rate limiting, and other fast-access data. Its low latency makes it ideal for data that's read or updated frequently. For rate limiting in a distributed system, I store request counters in Redis instead of application memory. Since all application instances share the same Redis server, every request updates the same counter regardless of which server handles it. I use an atomic operation—often with a Lua script—to increment the counter and set its expiration in one step, ensuring the rate limit remains accurate even under high concurrency.

CODE · TypeScript

```
import Redis from 'ioredis';
const redis = new Redis(process.env.REDIS_URL!);
// atomic fixed-window limiter: INCR then set TTL on first hit
const LUA = `
  local current = redis.call('INCR', KEYS[1])
  if current == 1 then redis.call('PEXPIRE', KEYS[1], ARGV[1]) end
  return current`;
export function rateLimit(limit: number, windowMs: number) {
  return async (req: any, res: any, next: any) => {
    const key = `rl:${req.ip}:${Math.floor(Date.now() / windowMs)}`;
    const count = (await redis.eval(LUA, 1, key, windowMs)) as number;
    if (count > limit) {

      res.setHeader('Retry-After', Math.ceil(windowMs / 1000));
      return res.status(429).json({ error: 'Too many requests' });
    }
    next();
  };
}
```

ARCHITECTURE / FLOW

```
Per-instance counters (WRONG): limit leaks x N
   Inst A --+
   Inst B --+--> [ Redis: INCR rl:ip:window ]  <- one truth
   Inst C --+        atomic INCR + PEXPIRE
                     count > limit -> 429
  Same limit enforced no matter how many instances run.
```

Q89 BACKEND & DATA

How do you handle concurrency, race conditions, and make write operations idempotent?

THEORY

A race condition happens when concurrent requests read-modify-write shared state and one overwrites the other, causing lost updates or double charges. You prevent it with atomic operations, conditional updates, or locks so only one writer wins. Idempotency means the same request applied twice has the same effect as once, which protects against client retries and network duplicates.

STRONG ANSWER

When dealing with concurrent requests, my goal is to ensure data stays correct even if multiple clients act at the same time. I rely on atomic database operations, optimistic locking when appropriate, and idempotency for operations that may be retried. For example, instead of reading a value and then updating it in separate steps, I perform the check and update in a single SQL statement so the database guarantees correctness. For operations like payments or order creation, I use an idempotency key. If the same request is retried because of a network failure, the server returns the original result instead of performing the operation again. For distributed coordination, I use Redis-based locks only when a critical section cannot be safely handled by the database.

CODE · TypeScript

```typescript
// idempotent create using a client-supplied key
export async function createPayment(req: any, res: any) {
  const key = req.headers['idempotency-key'] as string;
  if (!key) return res.status(400).json({ error: 'Idempotency-Key required' });
  const existing = await db.payments.findByIdemKey(key);
  if (existing) return res.status(200).json({ data: existing }); // safe retry
  // atomic balance check + decrement: DB enforces, no read-then-write race
  const debited = await db.query(
    `UPDATE accounts SET balance_cents = balance_cents - $1
     WHERE user_id = $2 AND balance_cents >= $1 RETURNING id`,
    [req.body.amount, req.body.userId]
  );
  if (debited.rowCount === 0) return res.status(409).json({ error: 'Insufficient funds' });
  const payment = await db.payments.insert({ idemKey: key, ...req.body });
  return res.status(201).json({ data: payment });
}
```

ARCHITECTURE / FLOW

```
Lost update (race):
  R1 read bal=100 --\
  R2 read bal=100 --/ both write 100-50 -> 50 (one lost!)
Atomic guarded UPDATE:
  UPDATE ... SET bal=bal-50 WHERE bal>=50  -> DB serializes
Idempotency:
  retry same key -> return stored result, no double charge
```

Q90 BACKEND & DATA

What are the basics of error handling, input validation, and API security you always apply?

THEORY

Every input is untrusted, so you validate and sanitize at the boundary with a schema before it reaches business logic, rejecting bad requests with a clear 400. Errors should flow into a single centralized handler that returns a consistent shape and never leaks stack traces or internal details. Core hardening includes CORS allow-lists, rate limiting, security headers, and not exposing whether a credential check failed on email vs password.

STRONG ANSWER

I follow a defense-in-depth approach. First, I validate every request—body, path parameters, and query parameters—using a schema validation library like Zod before the request reaches the business logic. Invalid input is rejected immediately with a 400 Bad Request. For error handling, I use a centralized Express error-handling middleware that returns consistent error responses to the client while logging detailed errors on the server. I never expose stack traces or sensitive information in production. For API security, I enable security headers with Helmet, configure CORS with an allow-list of trusted origins, apply authentication and authorization where needed, rate-limit requests using Redis to prevent abuse, and validate pagination and request sizes to avoid resource exhaustion.

CODE · TypeScript

```typescript
import { z } from 'zod';
import { Request, Response, NextFunction } from 'express';
const CreateUser = z.object({
  email: z.string().email(),
  age: z.number().int().min(13).max(120),
});
export function validate(schema: z.ZodTypeAny) {
  return (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({ error: 'Invalid input', issues: result.error.issues });
```

CODE · TypeScript (continued)

```
    }
    req.body = result.data; // sanitized, typed
    next();
  };
}
// centralized error handler: consistent shape, no leaks
export function errorHandler(err: any, _req: Request, res: Response, _next: NextFunction) {
  const status = err.status ?? 500;
  console.error(err); // full detail stays server-side
  res.status(status).json({ error: status === 500 ? 'Internal error' : err.message });
}
```

ARCHITECTURE / FLOW

```
Request
  -> [CORS allow-list] -> [rate limit] -> [validate schema]
        bad origin 403     too many 429     bad body 400
  -> [controller] --(throws)--> [central error handler]

        returns consistent JSON, logs detail, no stack leak
10
Payments, Releases, Testing & DevOps
Payments & webhooks, signing, staged rollout, Fastlane, CI/CD, tests, Sentry.
Q91–Q100
10
Payments, Releases, Testing & DevOps
Payments & webhooks, signing, staged rollout, Fastlane, CI/CD, tests, Sentry.
Q91–Q100
10
Payments, Releases, Testing & DevOps
Payments & webhooks, signing, staged rollout, Fastlane, CI/CD, tests, Sentry.
Q91–Q100

10
Payments, Releases, Testing & DevOps
Payments & webhooks, signing, staged rollout, Fastlane, CI/CD, tests, Sentry.
Q91–Q100
```

10 Payments, Releases, Testing & DevOps

Payments & webhooks, signing, staged rollout, Fastlane, CI/CD, tests, Sentry.

Q91–Q100

Q91 PAYMENTS, RELEASES, TESTING & DEVOPS

How do you integrate a payment gateway like Stripe or Razorpay into a React Native app safely?

THEORY

A payment flow should never trust the client to decide the amount or confirm success. The mobile app collects payment details through the gateway's SDK, but the price, currency, and order are always created on your server, which alone holds the secret API key. The client only receives a short-lived token or client secret to complete the charge.

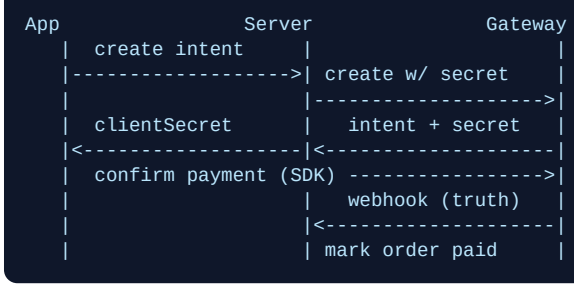
STRONG ANSWER

The key principle is that the backend owns the payment process, not the mobile app. The React Native app sends the cart or product information to the backend, and the backend calculates the final amount and creates a payment order or PaymentIntent with the payment provider. It then returns only the information the client needs, such as a client secret or order ID, to open the native payment sheet. After the user completes the payment, I don't trust the client callback to mark the payment as successful. Instead, I wait for a server-side webhook from Stripe or Razorpay, verify the payment, and only then update the order status in the database. This prevents users from tampering with payment amounts or faking successful payments.

CODE · TypeScript

```typescript
// client: ask server for a PaymentIntent, then confirm with the SDK
async function startCheckout(planId: string) {
  // server decides the real amount; client sends only the plan id
  const res = await api.post('/payments/intent', { planId });
  const { clientSecret } = res.data;
  const { error } = await confirmPayment(clientSecret, {
    paymentMethodType: 'Card',
  });
  if (error) {
    showError(error.message);
    return;
  }
  // optimistic UI only; truth comes from the webhook on the server
  navigate('PaymentProcessing', { planId });
}
```

ARCHITECTURE / FLOW



Q92 PAYMENTS, RELEASES, TESTING & DEVOPS

What are webhooks and why are they the source of truth for payment status instead of the client response?

THEORY

A webhook is a server-to-server HTTP callback the gateway sends directly to your backend when an event happens, such as a successful charge or a failed renewal. It is the source of truth because it comes straight from the gateway over a signed, authenticated channel, bypassing the user's device entirely. Client callbacks can be lost, faked, or interrupted, so they are only hints for the UI.

STRONG ANSWER

A webhook is an HTTP callback sent directly from the payment provider to our backend whenever a payment event occurs. It's considered the source of truth because it comes from the payment provider—not the client application—and can be verified using a cryptographic signature. I never rely on the client callback to update payment status because the app could crash, lose network connectivity, or be tampered with after the payment completes. Instead, I verify the webhook's signature, confirm the payment details, and only then update the order or subscription in the database.

CODE · TypeScript

```
import express from 'express';
import Stripe from 'stripe';
const stripe = new Stripe(process.env.STRIPE_KEY!);
// raw body is required so the signature can be verified
export const webhook = express.raw({ type: 'application/json' });
export async function handler(req: express.Request, res: express.Response) {
  const sig = req.headers['stripe-signature'] as string;
  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(
      req.body, sig, process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err) {
    return res.status(400).send('invalid signature'); // reject forgeries
  }
  await processEvent(event); // only now do we trust it
  res.json({ received: true });
}
```

ARCHITECTURE / FLOW

```
Gateway ===signed POST===> Your Server
            (out of band, never via the phone)
  client callback ----> UI hint only (may be lost)
  webhook        ----> DB write (source of truth)
```

Q93 PAYMENTS, RELEASES, TESTING & DEVOPS

How do you make webhook processing idempotent so you never double-charge or process the same event twice?

THEORY

Gateways guarantee at-least-once delivery, meaning the same webhook can arrive multiple times due to retries or timeouts. Idempotency means processing the same event repeatedly has the same effect as processing it once. The standard approach is to store each event's unique id and reject or no-op any id you have already handled, ideally inside a database transaction.

STRONG ANSWER

Payment gateways can retry the same webhook multiple times if they don't receive a successful response, so webhook handlers must be idempotent. I use the gateway's unique event ID as an idempotency key and store it in a processed_events table with a unique constraint or primary key. When a webhook arrives, I first try to record its event ID. If it already exists, I know the event has been processed before, so I immediately return 200 OK without running the business logic again. I perform both the event recording and the business update inside a single database transaction, ensuring the webhook is either fully processed or not processed at all.

CODE · TypeScript

```
async function processEvent(event: { id: string; type: string; data: any }) {
  await db.transaction(async (tx) => {
    // unique constraint on event_id makes this the dedup gate
    const inserted = await tx
      .insertInto('processed_events')
      .values({ event_id: event.id, type: event.type })
      .onConflict((c) => c.column('event_id').doNothing())
      .executeTakeFirst();
    if (Number(inserted.numInsertedOrUpdatedRows) === 0) {
      return; // already processed -> no-op, safe to ack
    }
    if (event.type === 'invoice.paid') {
      await extendSubscription(tx, event.data.subscriptionId, event.data.period);
    }
  });
}
```

ARCHITECTURE / FLOW

```
event id E1 arrives (1st time)
    INSERT E1 -> ok -> apply business logic -> ack 200
  event id E1 arrives again (retry)
    INSERT E1 -> conflict -> skip logic -> ack 200
                (same end state, no double-charge)
```

Q94 PAYMENTS, RELEASES, TESTING & DEVOPS

How do you handle the subscription lifecycle states like trial, active, past_due, and canceled?

THEORY

A subscription is a state machine the gateway drives through webhooks, and your job is to mirror that state and gate feature access on it. The common path is trialing then active, but a failed renewal moves it to past_due where the gateway retries the card, and after retries are exhausted it becomes canceled. Access should follow the entitlement, not the raw payment, so trial and past_due users may still keep access for a grace window.

STRONG ANSWER

I model subscriptions as a state machine, and I let server-side webhooks drive all state transitions. The client never decides the subscription status—it simply displays what the backend returns. During the trial and active states, the user has full access. If a renewal payment fails, the payment gateway changes the subscription to past_due. During this period, I usually keep access enabled while the gateway retries the payment (the dunning period), and I notify the user to update their payment method. If the retries still fail, the subscription moves to canceled. I don't revoke access immediately—I wait until the end of the current billing period so the user receives the service they've already paid for. One important lesson is to separate subscription status from access logic. For example, past_due doesn't necessarily mean the user should lose access immediately.

CODE · TypeScript

```typescript
type SubStatus = 'trialing' | 'active' | 'past_due' | 'canceled';
function hasAccess(s: { status: SubStatus; currentPeriodEnd: number }): boolean {
  const now = Date.now();
  switch (s.status) {
    case 'trialing':
    case 'active':
      return true;
    case 'past_due':
      return now < s.currentPeriodEnd; // grace window during dunning
    case 'canceled':
      return now < s.currentPeriodEnd; // keep until paid period ends
  }
}
// webhook handlers drive the transitions
const transitions: Record<string, SubStatus> = {
  'customer.subscription.trial_will_end': 'trialing',
  'invoice.paid': 'active',
  'invoice.payment_failed': 'past_due',
  'customer.subscription.deleted': 'canceled',
};
```

ARCHITECTURE / FLOW

```
trialing --invoice.paid--> active
     |                        |
     |                payment_failed
     v                        v
  active <--invoice.paid--- past_due --retries fail--> canceled
                            (grace)        (access til period end)
```

Q95 PAYMENTS, RELEASES, TESTING & DEVOPS

What goes into app signing and store compliance for shipping a React Native app to the App Store and Play Store?

THEORY

Both stores require every build to be cryptographically signed so they can prove it came from you and was not tampered with. Android uses a keystore (and Google Play App Signing) while iOS uses certificates and provisioning profiles tied to your Apple Developer account. Beyond signing, stores run compliance checks on permissions, privacy declarations, and metadata before the build goes live.

STRONG ANSWER

App signing ensures that only trusted builds of my app can be published and updated. On Android, I use an upload key together with Play App Signing, where Google securely manages the app signing key and I use my upload key to sign releases. On iOS, I use Apple distribution certificates and provisioning profiles, and I automate certificate management with Fastlane Match so the team can share them securely. For store compliance, I make sure the app meets the latest platform requirements before release. That includes providing accurate privacy disclosures, requesting only the permissions the app actually needs, completing the App Store Privacy Nutrition Label and Google Play Data Safety form, targeting the required Android API level, and following both stores' review guidelines. Proper signing protects the app, while compliance ensures the release is approved.

CODE · RUBY(FASTLANE)

```ruby
# Fastlane match: shared, encrypted iOS signing identities
lane :certificates do
  match(type: "appstore", readonly: true)
end
lane :compliance_check do
  # fails the build early if privacy metadata is missing
  ensure_no_debug_code(text: "console.log")
  upload_to_app_store(
    submit_for_review: false,
    precheck_include_in_app_purchases: true,
    force: true # use metadata files as the source of truth
  )
end
```

ARCHITECTURE / FLOW

```
iOS:  cert + provisioning profile -> signed .ipa -> App Store
          (managed via fastlane match, encrypted git repo)
  Android: upload key -> Play App Signing -> final signing key
          (Google holds final key; upload key recoverable)
  Store checks: permissions | privacy labels | target API | metadata
```

Q96 PAYMENTS, RELEASES, TESTING & DEVOPS

How do you manage a staged rollout and releases using Fastlane?

THEORY

A staged rollout releases a new version to a small percentage of users first, then widens the audience as confidence grows, so a regression hits 1 percent instead of everyone. Fastlane scripts the repetitive release steps such as building, signing, uploading, and setting the rollout fraction. The point is to catch crashes in the wild before full exposure and to be able to halt instantly.

STRONG ANSWER

I automate releases with Fastlane so every build follows the same repeatable pipeline instead of relying on manual steps. My Fastlane lanes handle building, code signing, uploading to the App Store or Google Play, and triggering staged rollouts. For production releases, I don't deploy to 100% of users immediately. I start with a small rollout, typically 1–5%, and monitor crash-free sessions, error rates in Sentry, and key business metrics. If everything looks healthy, I gradually increase the rollout to 10%, 50%, and then 100%. If I detect an issue, I pause or halt the rollout before it affects the entire user base. Automating this process makes releases consistent, safer, and much easier to reproduce.

CODE · RUBY(FASTLANE)

```ruby
# Fastlane: staged Play Store rollout with a parameterized fraction
lane :rollout do |options|
  fraction = options[:fraction] || 0.05 # start at 5%
  gradle(task: "bundle", build_type: "Release")
  upload_to_play_store(
    track: "production",
    rollout: fraction.to_s,
    aab: lane_context[SharedValues::GRADLE_AAB_OUTPUT_PATH],
    skip_upload_metadata: true
  )
  slack(message: "Rolled out v#{flutter_version} to #{fraction * 100}%")
end
# halt instantly if crash-free rate drops
lane :halt do
  upload_to_play_store(track: "production", rollout: "0")
end
```

ARCHITECTURE / FLOW

```
build+sign --> 1% --watch Sentry--> 10% --watch--> 50% --> 100%
                 |                     |
            crash spike?          crash spike?
                 |                     |
                 v                     v
              HALT (rollout = 0, no full exposure)
```

Q97 PAYMENTS, RELEASES, TESTING & DEVOPS

How would you set up CI/CD with GitHub Actions for a React Native mobile app?

THEORY

CI runs lint, type-check, and tests on every pull request so broken code never merges, while CD builds and ships signed artifacts on tagged releases. For mobile you need macOS runners for iOS builds and to inject signing secrets securely rather than committing them. Caching dependencies keeps the pipeline fast enough that people actually wait for it.

STRONG ANSWER

I set up CI/CD using GitHub Actions with two main pipelines: a fast PR validation pipeline and a release pipeline. For pull requests, I run a lightweight workflow that installs dependencies, runs ESLint, TypeScript checks, and unit tests with Jest. This acts as a quality gate before merging. I also use caching for node_modules, Gradle, and CocoaPods to keep feedback fast—ideally within a few minutes. For releases, I trigger workflows on Git tags. The iOS build runs on a macOS GitHub runner because Xcode is required, and Android runs on Linux. Signing credentials are stored securely in GitHub Secrets and injected at build time, never committed to the repository. Finally, I integrate Fastlane into the pipeline so the same lanes used locally are executed in CI. Fastlane handles building, signing, and uploading to the App Store and Google Play, which ensures consistency and removes environment-specific release issues.

CODE · YAML

```yaml
name: ci
on:
  pull_request:
  push:
    tags: ['v*']
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

CODE · YAML (continued)

```yaml
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: 'npm' }
      - run: npm ci

      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm test -- --ci --coverage
  ios-release:
    if: startsWith(github.ref, 'refs/tags/')
    needs: test
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: bundle exec fastlane ios release
        env:
          MATCH_PASSWORD: ${{ secrets.MATCH_PASSWORD }}
```

ARCHITECTURE / FLOW

```
PR opened
    -> install -> lint -> tsc -> jest   (gate merge)
  tag v*
    -> test passes
       -> ubuntu: build android aab
       -> macos:  build ios ipa (Xcode)
          -> fastlane -> stores (secrets injected, never in repo)
```

Q98 PAYMENTS, RELEASES, TESTING & DEVOPS

How do you write effective unit tests with Jest and React Native Testing Library?

THEORY

React Native Testing Library encourages testing behavior from the user's perspective rather than internal component state. You query elements the way a user finds them, by visible text or accessibility role, and assert on what renders, which makes tests resilient to refactors. Jest provides the test runner, assertions, and mocking for things like network calls and native modules.

STRONG ANSWER

I write tests using Jest and React Native Testing Library with a focus on testing user behavior rather than implementation details. Instead of accessing internal state or component methods, I query the UI the way a user would—using text, accessibility labels, and roles—and I trigger real user interactions like presses and input changes. I mock external dependencies such as network requests, APIs, and native modules to keep tests deterministic and fast. For asynchronous UI updates, I use findBy queries or waitFor to properly handle state changes after data loading. I also focus tests on critical business logic paths—like different subscription states or payment flows—so I cover scenarios where bugs would have real user impact, rather than trying to test every implementation detail.

CODE · TypeScript

```typescript
import { render, screen, fireEvent } from '@testing-library/react-native';
import { SubscriptionBanner } from './SubscriptionBanner';
describe('SubscriptionBanner', () => {
  it('prompts to update payment when past_due', () => {
    const onUpdate = jest.fn();
    render(<SubscriptionBanner status="past_due" onUpdate={onUpdate} />);
    // query the way a user would: by visible text / role
    expect(screen.getByText(/update your payment method/i)).toBeOnTheScreen();
    fireEvent.press(screen.getByRole('button', { name: /update/i }));
    expect(onUpdate).toHaveBeenCalledTimes(1);
```

CODE · TypeScript (continued)

```
  });
  it('shows nothing when active', () => {
    render(<SubscriptionBanner status="active" onUpdate={jest.fn()} />);
    expect(screen.queryByText(/update your payment/i)).toBeNull();

  });
});
```

Q99 PAYMENTS, RELEASES, TESTING & DEVOPS

How do you use Detox for end-to-end testing in a React Native app and how is it different from unit tests?

THEORY

Detox is a gray-box end-to-end framework that drives the real app on a simulator or device, tapping and typing like a user across whole flows. Unlike unit tests it exercises navigation, native modules, and the real bridge, so it catches integration bugs a component test cannot. Its key feature is being synchronization-aware: it waits for the app to be idle instead of using brittle fixed sleeps.

STRONG ANSWER

I use Detox for end-to-end testing of critical user journeys in a React Native app, such as onboarding, authentication, and payment flows. Unlike unit tests, Detox runs the actual app binary on a simulator or device and interacts with it like a real user would, using accessibility IDs to find elements and perform actions. A key advantage of Detox is that it is 'gray-box aware'—it understands when the app is busy with network requests, animations, or async work, so it automatically waits for the app to become idle instead of relying on manual delays, which reduces flakiness. I keep Detox tests focused on high-value flows like checkout or subscription purchase and run them in CI on a scheduled or nightly basis because they are slower and heavier than unit tests. Unit tests validate individual components and logic in isolation, while Detox validates the entire system from UI to backend integration.

CODE · TypeScript

```
describe('checkout flow', () => {
  beforeAll(async () => {
    await device.launchApp({ newInstance: true });
  });
  it('completes a subscription purchase in test mode', async () => {
    await element(by.id('plan-pro')).tap();
    await element(by.id('checkout-button')).tap();
    // Detox waits for the native payment sheet automatically
    await element(by.id('card-number')).typeText('4242424242424242');
    await element(by.id('pay-now')).tap();
    // assert the success screen rendered after the round trip
    await expect(element(by.text('Subscription active'))).toBeVisible();
  });
});
```

ARCHITECTURE / FLOW

```
unit (Jest+RNTL): one component, mocked deps, milliseconds
  e2e  (Detox):     whole app on simulator, real nav+native
   tap plan -> tap checkout -> type card -> pay
        \__________ real bridge + network ________/
                   -> assert 'Subscription active'
```

Q100 PAYMENTS, RELEASES, TESTING & DEVOPS

How do you set up monitoring and crash reporting with Sentry, and how do you read a crash report?

THEORY

Sentry captures unhandled crashes and errors with full stack traces, breadcrumbs, device context, and the app release, so you can diagnose issues you cannot reproduce locally. For React Native you upload source maps and native symbols so minified stack traces map back to your real code. A crash report's value is in its context: the breadcrumbs leading up to the error and which release and device it hit.

STRONG ANSWER

I set up Sentry at app startup with the correct release version and environment (development, staging, production). In CI, I upload source maps for JavaScript and dSYMs for iOS so that stack traces are properly symbolicated—otherwise crashes are unreadable minified code. When analyzing a crash, I start from the exception type and message, then look at the stack trace to identify whether it originates from our application code or a dependency. After that, I use breadcrumbs to understand the user's last actions leading up to the crash, such as navigation events, API calls, or UI interactions. I also check the release version, device model, and OS version to understand the scope of impact. During staged rollouts, I primarily monitor crash-free session rate per release. If I see a spike, I immediately pause or roll back the rollout. Breadcrumbs are often the most useful part because they help reproduce the exact user flow that caused the issue.

CODE · TypeScript

```
import * as Sentry from '@sentry/react-native';
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  release: `app@${appVersion}`, // ties crashes to a staged rollout
  environment: __DEV__ ? 'dev' : 'production',
  tracesSampleRate: 0.2,
  beforeSend(event) {
    // never ship card data to the crash reporter
    delete event.extra?.cardNumber;
    return event;
  },
});
// add context so a crash report is actually readable later
Sentry.addBreadcrumb({ category: 'payment', message: 'confirm tapped' });
Sentry.setTag('subscription_status', 'past_due');
```

ARCHITECTURE / FLOW

```
Crash report, read top-down:
  +-----------------------------------------+
  | TypeError: undefined is not an object     | <- what
  | at CheckoutScreen.tsx:42 (your frame)     | <- where
  | breadcrumbs: view plan > confirm tapped   | <- how
  | release app@2.3.0  iOS 17.4  device X     | <- who/scope
  +-----------------------------------------+
   spike in crash-free rate -> halt rollout
```